Collaborative Active Learning Based on Improved Capsule Networks for Hyperspectral Image Classification

Heng Wang and Liguo Wang, *Member, IEEE*

***Abstract*— For hyperspectral image classification (HIC) tasks, most uncertainty-based active learning (AL) methods only consider the uncertainty, without considering the diversity of actively selected samples and the budget of expert labeling. In this article, we propose a collaborative AL (CAL) framework to address this problem. The proposed framework consists of two well-designed base classifiers and an ingenious CAL scheme that takes into account both the uncertainty and diversity of actively selected samples and the cost of expert annotation. Specifically, getting benefit from the ability of capsule networks (CapsNets) to accurately identify and locate features, we design two improved CapsNets. For these two networks, we call the first one Capsule Vision Transformer (CapsViT), which introduces vision Transformer (ViT) into the CapsNet to learn the global relationship between the capsule features. We call the second one Capsule GLOM (CapsGLOM); the basic structure of this network is derived from the GLOM system proposed by Geoffrey Hinton; and we learn from the way CapsNet constructs the primary capsules to improve its implementation details. CapsViT and CapsGLOM are used as the two base classifiers in the proposed CAL framework to select the most informative samples according to the CAL scheme under the premise of fully considering the cost of expert annotation. Experimental results on four benchmark hyperspectral image datasets show that our proposed CAL framework can achieve satisfactory classification results. At the same time, compared with other advanced deep models, our proposed CapsViT and CapsGLOM are also competitive in the supervised HIC tasks. The source code is available online (https://github.com/swiftest/CAL).**

***Index Terms*— Capsule GLOM (CapsGLOM), capsule vision Transformer (CapsViT), collaborative active learning (CAL), hyperspectral image classification (HIC), vision Transformer (ViT).**

I. Introduction

HYPERSPECTRAL images (HSIs) obtained by hyperspectral sensors mounted on airborne and spaceborne aircraft record ground scenes with hundreds of continuous spectral bands. Although the spatial resolution of HSIs is usually low, this multispectral imaging characteristic makes it widely used in precision agriculture, mineral detection, and urban development planning [1], [2], [3]. HSI classification (HIC), as a basic and important task in remote sensing, has always been an active topic in the community. A large number of spectral bands imply more classification information, but this may lead to the so-called Hughes phenomenon and higher computing costs. In order to reduce spectral redundancy, some dimensionality reduction methods, such as traditional principal component analysis (PCA) [4], band selection [5], and automatic encoder based on the 3-D convolution neural network (CNN) [6], are used. These methods are effective and successful, but they inevitably lose some information to some extent. Some recent studies [7], [8], [9], [10] have retained full band information when designing classification models, as these models have a sufficiently large model capacity.

The two most severe challenges HIC faces are given as follows: 1) high intraclass difference and interclass similarity and 2) the scarcity of labeled training samples. For the first challenge here, with the rapid development and continuous progress of deep learning (DL) in the field of computer vision (CV) and natural language processing (NLP), many excellent deep models aimed at extracting deep discriminable features have been developed. Roy et al. [4] constructed hybrid spectral CNN (HybridSN), a simple and efficient network, using the sequential arrangement of 3-D and 2-D CNNs. Soon after, Roy et al. [10] designed a novel attention-based adaptive spectral–spatial kernel residual network (A2S2K-RN); the enlightening part of this model is that it uses $1 \times 1 \times 7$ and $3 \times 3 \times 7$ 3-D convolution kernels to jointly learn adaptive spectral–spatial features. Although the pooling operation in CNNs can effectively detect whether a certain feature appears, in order to more accurately model the relative position relationship of the emerging features, Paoletti et al. [11] first introduced capsule network (CapsNet) into HIC to achieve more accurate classification. The 3-D octave convolution with the spatial–spectral attention network (3DOC-SSAN) proposed by Tang et al. [12] has similar advantages to the SpectralNET proposed by Chakraborty and Trehan [13]. The former uses four 3-D octave convolution blocks to capture high- and low-frequency information in the spatial domain, while the latter performs four-level wavelet decomposition on the input patch to explore high- and low-frequency spatial information. From the perspective of grouping the spectral sequence into input signals of different time steps, many deep models based on recurrent neural networks (RNNs) have been studied and applied to HIC tasks [8], [14], [15].

Manuscript received 12 May 2023; revised 8 August 2023; accepted 24 August 2023. Date of publication 28 August 2023; date of current version 11 September 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62071084 and in part by the Leading Talents Project of the State Ethnic Affairs Commission. *(Corresponding author: Liguo Wang.)*

The authors are with the College of Information and Communications Engineering, Harbin Engineering University, Heilongjiang, Harbin 150001, China (e-mail: wanghenghrbeu@hrbeu.edu.cn; wangliguo@hrbeu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2023.3309269

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

Hong et al. [16] proposed a minibatch graph convolution network (miniGCN) so that GCN can be trained in a minibatch fashion on large-scale remote sensing datasets. With the emergence of Transformer in the field of NLP and its rapid dominance [17], Dosovitskiy et al. [18] pointed out that an image is equivalent to 16×16 words and proposed the vision Transformer (ViT). Inspired by ViT's powerful learning ability for global contexts, He et al. [19] sent the underlying features extracted from a VGG-like deep CNN to the innovatively designed DenseTransformer module to learn long-range spectral dependencies. Hong et al. [20] designed a backbone network called SpectralFormer and a cross-layer adaptive fusion (CAF) module, which is used to fuse the Transformers' output at different depths to improve the information exchange between different layers. Zhang et al. [6] introduced MobileViT to build a lightweight classification model. Zhao et al. [21] built a convolution Transformer network (CTN) to capture the local–global characteristics of HSI patches. Wang et al. [22] proposed a dual context network based on Transformer encoder and decoder (DCN-T), which uses Transformer to effectively encode regional adaptation and global aggregation spatial contexts within and between homogeneous regions discovered based on similarity clustering. Sun et al. [23] constructed a spectral–spatial feature tokenization transformer (SSFTT) model to capture spectral–spatial features and high-level semantic features. Ravikumar et al. [24] constructed a novel model based on the concept of deep matrix CapsNet (DMCapsNet) with an expectation-maximization (EM) routing algorithm. The model first calculates the votes of convolutional capsules by multiplying the primary capsules (PrimaryCaps) by the viewpoint invariant transformation matrix and then calculates the Gaussian mixture distributions of convolutional capsules by the assignment probabilities and votes. Finally, based on the similarities between the votes and distributions, the PrimaryCaps are aggregated into the convolutional capsules to realize the modeling of the part-whole relationship. As the core algorithm of the model, the EM algorithm [25] updates the Gaussian distributions' parameters according to the activations, votes, and assignment probabilities at step M and updates the assignment probabilities according to the similarity between the votes and updated Gaussian distributions at step E.

For the second challenge, there are few labeled training samples. Although grid search can be used to train a relatively optimal model, this problem greatly limits various deep models to give full play to their potential. Semisupervised HIC is widely investigated as a research hotspot, and the core problem is how to make full use of and play the value of a large number of unlabeled samples [26], [27], [28], [29]. Wu and Prasad [30] proposed the most classical semisupervised classification method using pseudolabels, in which a deep convolutional RNN (CRNN) model was pretrained using all the training data together with their pseudolabels that were defined as cluster assignments generated by the constrained Dirichlet process mixture model (C-DPMM) and then fine-tuned using the limited available labeled data. Fang et al. [31] put forward a collaborative learning framework that first trains a lightweight CNN model with the labeled samples and then uses the approximate rank-order clustering (AROC) to cluster deep features of all samples, and finally, the unlabeled samples whose clustering labels are consistent with the predicted labels are used to expand the training set to conduct iterative training. Seydgar et al. [32] developed a probabilistic pseudolabel generation framework for semisupervised HIC. This framework assumes that the encoded features of (a certain class) labeled samples obey a Gaussian distribution and then uses the maximum likelihood estimation (MLE) to estimate this distribution and assign the corresponding class label. When assigning labels to unlabeled samples, the framework uses a confidence interval to measure the similarity scores between the encoded features of unlabeled samples and the distribution of labeled samples. Zeng et al. [33] proposed a graph clustering convolution network, which uses biclustering to construct graphs to help GCN find reliable dependencies. Jia et al. [34] proposed a novel semisupervised Siamese network, which integrates a 3-D automatic encoder for learning potential deep features of all samples and a Siamese network for rectifying unsupervised features to reduce the intraclass distance and increase the interclass distance. Even though the above semisupervised classification methods based on the deep models are effective and the computational complexity is acceptable, the classification accuracy is often not satisfactory.

When the limited training samples are available, another solution is to introduce generative adversarial networks (GANs) [35], [36], [37], [38], [39], [40], [41]. Zhu et al. [42] used GAN in HIC based on auxiliary classifier GAN (AC-GAN) model to mitigate overfitting and improve classification performance. Zhan et al. [43] designed a 1-D GAN named HSGAN for semisupervised HIC. It uses unlabeled samples to train 1-D GAN; once the whole model's training is completed, the network is converted into a classifier by replacing the top layer of the discriminator with the softmax layer. Feng et al. [44] designed an enlightening convolution GAN based on cooperative learning and attention mechanism, in which the deep features of the real samples extracted by the discriminator are used to guide the generator to generate high-quality samples. For the sake of improving the diversity of samples generated by GAN and achieving high-quality restoration of the variational autoencoder (VAE), many combination models combining the advantages of these two generative models have been proposed, such as semisupervised variational GAN (SSVGAN) [45], conditional VAE with an adversarial training process (CVA2E) [46], self-attention-based conditional VAE GAN (SACVAEGAN) [47], and variational GAN with crossed spatial and spectral interactions (CSSVGAN) [48]. Wang et al. [49] used the CapsNet joint convolutional long short-term memory (ConvLSTM) module to design the discriminator, and constructed an AC-GAN model for HIC. Because the CapsNet has the ability to accurately locate and identify features, we can also see the models combining CapsNet and GAN in the literature [50], [51]. Some other interesting GAN-based classification models can be found in [52], [53], [54], [55], and [56]. In fact, the generator in the GAN model is an effective regularization method. However, the training of GAN is unstable, that is, it is difficult to achieve Nash equilibrium and prone to model collapse [39], [57], [58].

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

With limited training samples, ensemble learning [59], transfer learning [60], and domain adaptation (DA) [or domain generalization (DG)] [61], [62] also perform well. Traditional ensemble learning frameworks need to train several base networks, which greatly increases the computing cost. Lately, Xu et al. [63] proposed a robust self-ensembling network (RSEN) for HIC, which includes two subnetworks: the base network and the ensemble network with identical structures. Moreover, to make the pseudolabels of unlabeled samples predicted by the ensemble network more credible, the authors also proposed a consistency filtering strategy. The effectiveness of transfer learning lies in that the prior information learned in the auxiliary domain can be used to aid in learning new feature problems in the target HSI domain. For example, Hao et al. [64] used the pretrained residual network's (ResNet) knowledge about the different structures to formulate the geometry-aware ResNet loss in their proposed geometry-aware deep RNN (Geo-DRNN), so as to better encode the complex geometries. DA is designed to reduce domain shift at the feature level and learn a domain invariant model, such as the topological structure and semantic information transfer network (TSTnet) proposed by Zhang et al. [65], which uses graph optimal transport (GOT) to align topological relationships between source domain (SD) and target domain (TD) subgraphs to assist distribution alignment. DG is more challenging than DA, as it learns the model only on the SD data and does not access the TD during the training phase. Recently, Zhang et al. [66] proposed a DG framework for cross-scenario HIC, called single-SD expansion network (SDEnet). The generator of the framework uses a semantic encoder to ensure that the extended domain (ED) contains specific information about various TDs and a morph encoder to extract domain invariant features. The framework's discriminator uses supervised contrastive learning to learn classwise domain invariant representation between SD, ED, and their random linear combination intermediate domain (ID).

Active learning (AL) considers the HIC problem when there are only a few labeled samples from another perspective. Unlike methods that use pseudolabels, AL constructs the optimal training dataset by querying the most informative samples and supplemented by expert annotation [67], [68], [69]. Cao et al. [2] presented an active DL approach in which candidate samples with high prediction entropy (EP) values or low best versus second best (BvSB) values were selected for active labeling, and the Markov random field (MRF) was utilized to enforce class label smoothness during iterative training. Liu et al. [70] proposed an AL scheme based on a weighted incremental dictionary learning, which projects the initial training data to the topmost output of the deep belief network (DBN) (with the unsupervised learning stage) as the initial dictionary atoms and then selects some of the best samples for labeling in each iteration according to the representative measurement based on sparse representation and the uncertainty measurement based on information entropy. Haut et al. [71] used the average posterior probability predicted by the developed Bayesian CNNs (B-CNNs, dropout networks) to measure the uncertainty of samples in the candidate pool, and analyzed and compared the performance of different AL acquisition functions in detail. Patel et al. [72] proposed an AL selection scheme based on diversity and uncertainty, which combines diversity criteria and BvSB criteria to actively select informative samples. First, they use the K-means algorithm to cluster the current training samples, then calculate the distance from the uncertain samples determined by BvSB to each cluster center, and take the cluster center with the minimum distance as the cluster label of these samples. Finally, samples whose distance to the centroid of the specified cluster is greater than the average distance are selected as samples with diversity. Wang et al. [73] introduced the random multigraph (RMG) algorithm and used the anchor graph algorithm (used to construct and learn graph) to predict the labels of uncertain samples determined by BvSB instead of the expert mark. Wang and Ren [74] proposed a feature-oriented adversarial AL (FAAL) framework, which uses GAN to learn the distribution of high-level features of labeled samples obtained by the convolution module in the classifier and then uses a well-trained discriminator as the acquisition heuristic to judge whether an unlabeled sample has been previously well represented or not. Yoo and Kweon [75] proposed a new selection strategy, namely, loss prediction, which uses the knowledge of multiple middle layers of the deep model to predict the target loss of unmarked input. Lei et al. [76] used ResNet-18 as the basic learner and loss prediction as the query function to investigate active DL for HIC. Liu et al. [77] used the middle layers of the deep densely connected CNN to construct the loss prediction module.

In general, a lot of labeled samples may be redundant and unnecessary because there may be duplicate samples that provide similar or identical information [34]. Although AL relies on prior expert knowledge or requires a certain cost of manual annotation, it may be the most practical, efficient, and effective method for HIC problems based on deep models. The uncertainty-based AL methods are still mainstream because they can improve the classifier performance rapidly in several iterations. However, most of these methods at this stage do not take into account the diversity of samples that need to be labeled actively except for document [72], and most of them do not take into account the budget of expert annotation except for document [74]. To address the above problems, this article develops a collaborative AL (CAL) framework with practical application significance for HIC. In detail, the framework includes two robust basic classifiers that perform CAL and a CAL scheme. One of the two robust classifiers is the CapsNet combined with ViT [Capsule ViT (CapsViT)], which resorts to the ability of CapsNet to accurately classify when limited labeled samples are available and the ability of ViT to learn the global correlation of features. Another basic classifier is inspired by the GLOM system proposed by Hinton [78]. This is a powerful system, but some ideas for building this system cannot make it work perfectly. We improved it to apply to HIC and named it Capsule GLOM (CapsGLOM); as a robust basic classifier, it can learn the global dependencies of each location by constructing a parse tree. Besides, a simple and effective CAL scheme that considers both uncertainty and diversity, as well as an expert annotation budget, is proposed.

All in all, the motivation of this study is to address the problem that uncertainty-based AL methods fail to take into

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

account the diversity of actively selected samples and the cost of expert annotation; the main contributions are reflected in the following four aspects.

1) A novel CAL framework is devised for HIC with few labeled samples. The proposed framework consists of two robust base classifiers and a CAL scheme that considers both the diversity of the selected informative samples and the budget of expert annotation.
2) The CapsNet combined with ViT is developed as one of the two base classifiers in the proposed CAL framework. It makes full use of the ability of CapsNet to accurately locate and identify features and the unparalleled ability of ViT in global relevance learning.
3) Based on the improved GLOM system, CapsGLOM, a model with strong explanatory power, is proposed as another base classifier and applied to HIC for the first time.
4) We validate the effectiveness and advantages of our proposed framework by conducting rigorous experiments on four benchmark HSI datasets. At the same time, under the supervised classification scenario with sufficient training samples, it is verified that the proposed CasViT and CapsGLOM also obtain competitive performance compared with the current advanced deep models.

The subsequent text is organized as follows. Section II briefly introduces some related works, including the AL methods, CapsNet, ViT, and the GLOM system. Section III describes our proposed CAL framework, including the developed CapsViT, CapsGLOM, and CAL scheme. Section IV verifies the effectiveness and superiority of the proposed framework on four HSI datasets. Section V discusses the effectiveness, sensitivity, and complexity of the developed CapsViT and CapsGLOM. Section VI summarizes the work of the full text and puts forward some suggestions on the possible future research directions.

II. Related Works

This section briefly reviews the works related to the AL methods, CapsNet, ViT, and the GLOM system, providing a basis for the proposed CAL framework depicted in Section III.

A. AL Methods

AL is a powerful technique that advocates data efficiency and aims to achieve a certain level of prediction accuracy with the least amount of acquisition steps (oracle queries). AL methods can be roughly divided into four types: uncertainty-, representation-, performance- [67], and loss prediction-based AL [75]. Different AL methods use different query strategies, but the common goal of these heuristics is to search for the most informative samples.

Uncertainty-based methods select samples for which the deep model hesitates between two or more classes, and the most commonly used query strategies include BvSB, EP, Monte Carlo (MC) dropout, and so on. Although the uncertainty-based methods offer major advantages, such as their scalability to large datasets and their few hyperparameters, if the dataset contains many (close) replicas for each sample, they would select all replicas of a single informative sample at the expense of other informative samples, thus wasting data efficiency. It should be mentioned here that the Batch Bayesian AL by Disagreement (BatchBALD) proposed by Kirsch et al. [79] belongs to the method based on uncertainty and acquires points that are jointly informative but not individually informative (i.e., enforces diversity within a batch). However, BatchBALD extends to BALD based on Bayesian neural networks (BNNs), while our work focuses on backpropagation (BP) neural networks, where parameters are treated as point estimates rather than distributions.

Representation-based approaches attempt to query points to have the labeled points homogeneously spread throughout the whole data space. The most representative of these methods include the core-set approach [80] and variational adversarial AL (VAAL) [81]. Among them, the core-set approach is equivalent to solving the k-center problem, that is, selecting a batch of center points (labeled points) such that the largest distance between a data point (an unlabeled point) and its nearest center is minimized. The k-center problem is NP-hard; however, a 2-optimal (2-OPT) solution can be efficiently obtained using a greedy algorithm. The result of the greedy algorithm gives a good initialization, and Sener and Savarese [80] formulated a mixed integer program (MIP) to improve the 2-OPT solution. While the results of the core-set approach are impressive, when working with large datasets, calculating the distance between each candidate point and each training point is too time-consuming to be useful in practice. VAAL is a novel AL method, and the key to VAAL is that the VAE is used to map the labeled and unlabeled data into a latent feature space and trick the adversarial network (discriminator) into predicting that all data points, from both the labeled and unlabeled pools, are from the labeled pool, while the discriminator learns to effectively estimate the probability that the data come from the unlabeled data. Instead of explicitly measuring uncertainty, VAAL aims to query points that would yield high uncertainty and, thus, points that are not well represented in the labeled set. Although VAAL is effective, there are still questions about the stability and ease of training.

Performance-based methods attempt to automatically select the best query strategy by reliably estimating the improvement of the classifier's performance. There are two major limitations to these methods. First, they can only be combined with existing hand-designed heuristics, and second, reliable assessment of the classification performance from scarce annotated data is problematic. To overcome these limitations, Konyushkova et al. [82] proposed learning AL (LAL). The essence of LAL is a regression problem; that is, given a trained classifier and a candidate point, it estimates the reduction in generalization error that can be expected by adding the label to that point. Specifically, for each AL step t, LAL randomly splits the labeled points L_t into a training set L_{τ_i} and a test set U_{τ_i} according to different split sizes $\tau_i = 2, \ldots, T + 2$. For each split, M different points are sampled from U_{τ_i} and added to L_{τ_i} in turn, and the corresponding classifier/data point state pairs with an associated reduction in error are recorded. This process is repeated Q times, producing QMT records in total. Next, a regressor is trained to correlate changes in test performance with the properties of the classifier and the newly added data points. Finally, the obtained regressor is used as an

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

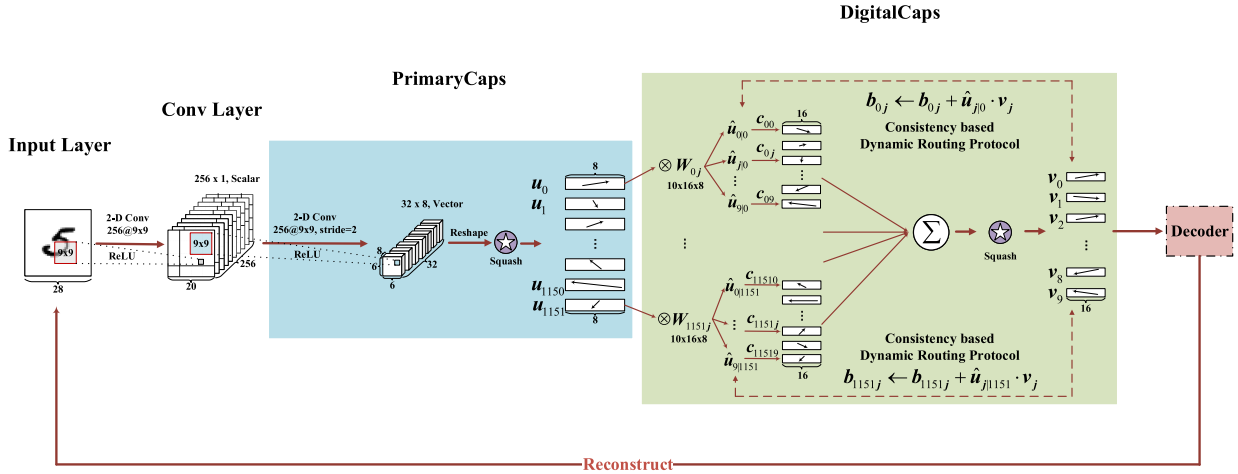


Fig. 1. CapsNet for handwritten character classification.

LAL strategy to query the data points with the highest potential error reduction. Although the idea of the LAL method is good, its complexity is often unacceptable, especially when a deep model is used as the classifier.

Loss prediction-based AL methods are simple but task-independent and work effectively with deep models. Here, we give a brief introduction to it so that we can compare different query strategies in the subsequent experiment section. This query strategy estimates the losses of unlabeled samples through a loss prediction module, which is connected to several layers of the target model and takes multilevel knowledge into consideration for loss prediction. Yoo et al. [75] positioned the loss function of the loss prediction module as

$$L_{\text{loss}}(\hat{l}^p, l^p) = \max\left(0, -\mathbb{1}\left(l_i, l_j\right) \cdot \left(\hat{l}_i - \hat{l}_j\right) + \xi\right) \tag{1}$$

where $l^p = (l_i, l_j)$ represents the losses of the training sample pair (x_i, x_j) calculated by the target model, $\hat{l}^p = (\hat{l}_i, \hat{l}_j)$ represents the losses calculated by the loss prediction module, ξ is a predefined positive margin (usually 1), and $\mathbb{1}(l_i, l_j)$ is an indicator function

$$\mathbb{1}(l_i, l_j) = \begin{cases} +1, & \text{if } l_i > l_j \\ -1, & \text{otherwise.} \end{cases} \tag{2}$$

When $l_i > l_j$, there is no loss given to the module only if $\hat{l}_i$ is larger than $\hat{l}_j + \xi$; otherwise, a loss is given to the module to force it increase $\hat{l}_i$ and decrease $\hat{l}_j$. For this heuristic, the greater the predicted loss of a sample, the more likely the classifier will misclassify it. As is seen, the calculation of the loss of this module requires that the batch size should be an even number. In the following experimental section, we will make a small modification to meet the situation that the batch size is not an even number.

In an earlier study, Demir et al. [83] combined the uncertainty and diversity criterion and investigated different AL techniques for the classification of remote sensing images with support vector machines (SVMs). Specifically, they first filter the unlabeled samples based on the uncertainty criterion and then select samples to label using a query function designed to assess the diversity. The proposed CAL framework also takes into account both uncertainty and diversity criteria. However, we have not defined any diversity criterion, and most importantly, we have emphasized the budget of expert annotation.

B. Capsule Networks

In CNN, although the pooling operation can identify specific features by virtue of "translation invariance," the spatial hierarchical relationship between these features fails to be considered. Compared with the invariance of CNN, Hinton proposed the concept of "equivariance"; that is, after the representation of an object is transformed, its result is equivalent to the representation of the transformed object. Based on the idea of "equivariance," the CapsNet is designed to accurately locate and recognize features. Fig. 1 shows the schematic of the classic CapsNet for handwritten character classification.

To start with, the input handwritten characters are fed into the first two convolution layers, and the activation function employed is the rectified linear unit (ReLU). At this point, the output is a feature cube with the shape of $256 \times 6 \times 6$. Each capsule is a "vector neuron" formed by the combination of a group of scalar neurons; therefore, from the perspective of building PrimaryCaps with a dimension of 8, the output of the second convolution layer can be regarded as a stack of 32 feature cubes with $8 \times 6 \times 6$ shape. Each feature cube consists of 36 (6×6) capsules, so the 32 feature cubes contain 1152 PrimaryCaps in total.

Each capsule vector reflects the representation of a specific entity. The mold of the vector describes the probability of the existence of the entity, and its direction corresponds to the general pose of the entity. In order to make the mold of the capsule vector better represent the existence of entities, it is necessary to apply a nonlinear squeeze function to ensure that the direction of the vector remains unchanged and, at the same time, compress the value of the mold to the [0∼1) interval. This is the functionality of the Squash function shown in Fig. 1; its formula is given as follows:

$$\text{Squash}(\boldsymbol{s}_j) = \frac{\|\boldsymbol{s}_j\|^2}{1 + \|\boldsymbol{s}_j\|^2} \cdot \frac{\boldsymbol{s}_j}{\|\boldsymbol{s}_j\|} = \frac{\|\boldsymbol{s}_j\|^2}{1 + \|\boldsymbol{s}_j\|^2} \hat{\boldsymbol{s}}_j \tag{3}$$

where $\hat{\boldsymbol{s}}_j = (\boldsymbol{s}_j / \|\boldsymbol{s}_j\|)$ is the unit vector of the input vector $\boldsymbol{s}_j$. The PrimaryCap output through the Squash function is expressed as $\boldsymbol{s}_i (i = 0, 1, 2, \ldots, 1151)$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

CapsNet uses many PrimaryCaps to represent all possible shallow entities, and each shallow entity can make category predictions for entities in the category capsule layer. The category capsule layer in the CapsNet is collectively called the digital capsule (DigitalCap) layer. There are ten capsule units that are represented by $\boldsymbol{v}_j (j = 0, 1, \ldots, 9)$, exactly corresponding to ten numbers to be recognized. Naturally, the prediction from the PrimaryCap to the DigitalCap is realized by using fully connected (FC) layer, and there is a coupling coefficient c_{ij} between each PrimaryCap $\boldsymbol{u}_i$ and each DigitalCap $\boldsymbol{v}_j$, which is a probability distribution and represents the support degree of the PrimaryCap to the DigitalCap. For each PrimaryCap, the sum of its coupling degree to all DigitalCaps is 1, that is,

$$\sum_j c_{ij} = 1. \tag{4}$$

The DigitalCap $\boldsymbol{v}_j$ is obtained by multiplying the prediction vector of each PrimaryCap relative to the DigitalCap by the corresponding coupling coefficient c_{ij} and then adding them together. Of course, do not forget to use the Squash function for compression activation. The whole process can be expressed as

$$\boldsymbol{v}_j = \text{Squash}\left[\sum_i c_{ij}\left(\boldsymbol{W}_{ij}\boldsymbol{u}_i\right)\right] = \text{Squash}\left(\sum_i c_{ij}\hat{\boldsymbol{u}}_{j|i}\right) \tag{5}$$

where $\hat{\boldsymbol{u}}_{j|i}$ denotes the prediction vector of the PrimaryCap $\boldsymbol{u}_i$ relative to the DigitalCap $\boldsymbol{v}_j$ and c_{ij} represents the coupling coefficient between them. It is interesting to note that c_{ij} is not a trainable parameter of the CapsNet but a parameter that is iteratively updated by a dynamic routing protocol based on consistency in the forward propagation. Specifically, it is determined by calculating the consistency b_{ij} between the DigitalCap $\boldsymbol{v}_j$ and the prediction vector $\hat{\boldsymbol{u}}_{ij}$ and then by the softmax function, which can be expressed as

$$c_{ij} = \frac{\exp(b_{ij})}{\sum_k \exp(b_{ik})} \tag{6}$$

where k denotes the number of categories of DigitalCap. At the initial time, the consistency between the DigitalCap and the prediction vector is initialized to zero vector, that is, $b_{ij} = 0$. Then, in each routing addressing process, consistency b_{ij} is updated in a cumulative way

$$b_{ij} \leftarrow b_{ij} + \hat{\boldsymbol{u}}_{j|i} \cdot \boldsymbol{v}_j \tag{7}$$

where $\hat{\boldsymbol{u}}_{j|i} \cdot \boldsymbol{v}_j$ represents the dot production between the prediction vector and the DigitalCap, which is the so-called consistency between them.

In order to control the learning process and determine multiple numbers (or categories) in an image, CapsNet calculates the margin loss for each DigitalCap $\boldsymbol{v}_j$ and sums them as the objective function of model optimization. For any DigitalCap $\boldsymbol{v}_j$, its margin loss can be expressed as

$$L_j = T_j \max(0, m^+ - \|\boldsymbol{v}_j\|)^2 + \lambda(1 - T_j)\max(0, \|\boldsymbol{v}_j\| - m^-)^2 \tag{8}$$

where T_j is the indicator function of category j; if and only if category j appears, $T_j = 1$; otherwise, $T = 0$. $\|\boldsymbol{v}_j\|$ represents the mold of $\boldsymbol{v}_j$, which is actually the probability of the existence of category j. m^+ is the upper margin threshold (usually 0.9) when category j appears; if its prediction probability is less than 0.9 (false negative), the model will be punished. Similarly, m^- is the lower margin threshold (usually 0.1) when category j does not appear; if its prediction probability is greater than 0.1 (false positive), the model will also be punished. λ is the sparse coefficient used to adjust the proportion of two losses that penalize false negative and false positive. To prevent excessive compression of the DigitalCap that does not appear at the beginning of learning, it is usually taken as 0.5.

The decoder in Fig. 1 can be any simple network, which is used to decode the DigitalCap corresponding to the label of the sample to reconstruct the input sample. Hence, to make the CapsNet also have advanced encoding and decoding functions, the loss function of the entire network can be written as

$$L = \frac{1}{N}\sum_{n=1}^{N}\left[\left(\sum_j L_{nj}^{\text{margin}}\right) + \alpha L_n^{\text{recon}}\right] \tag{9}$$

where L_{nj}^{margin} represents the margin loss of training sample n relative to the DigitalCap $\boldsymbol{v}_j$; L_n^{recon} represents the reconstruction loss, which can be calculated using root mean square error (RMSE); α denotes a regularization coefficient; owing to that the network focuses on classification tasks, it usually takes a small value, such as 0.01; and N denotes the size of the minibatch.

C. Vision Transformer

Transformer [17] significantly improved the current level of neuron machine translation (NMT) without using any cycle layer or convolution layer and only using the attention mechanism. In view of Transformer's ability to encode the relationship between each word and other words in the same sentence and pay more attention to the most relevant words, Dosovitskiy et al. [18] proposed ViT, and the model is shown in Fig. 2.

The standard Transformer receives a 1-D sequence of token embeddings as input; to process 3-D images, ViT first reshapes the input image $\boldsymbol{x} \in \mathcal{R}^{3\times H\times W}$ (taking the natural image as an example here) into a sequence of flattened patches $\boldsymbol{x}' = (\boldsymbol{x}_p^1, \boldsymbol{x}_p^2, \ldots, \boldsymbol{x}_p^N)^{\text{T}} \in \mathcal{R}^{N\times(3\cdot P^2)}$, where (H, W) is the resolution of the original image, (P, P) is the resolution of each patch, $\boldsymbol{x}_p^n \in \mathcal{R}^{3\cdot P^2}$ is the nth flattened patch, and $N = HW/P^2$ is the resulting number of patches. Next, these flattened patches $\boldsymbol{x}_p^n (n = 1, 2, \ldots, N)$ are fed into a linear layer without activation function to get the underlying embeddings. These underlying embeddings need to add a position embedding before being fed into the Transformer encoder. This allows the model to access the location of each patch, which is necessary because the encoder's "multihead attention" layer does not consider the order or location of patches but only their relationship. To perform the classification task, a category token needs to be inserted before all patches' embeddings and assigned a position embedding.

The Transformer encoder is composed of a multihead self-attention (MSA) layer and a small multilayer perceptron (MLP), both of which are based on the residual structure.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

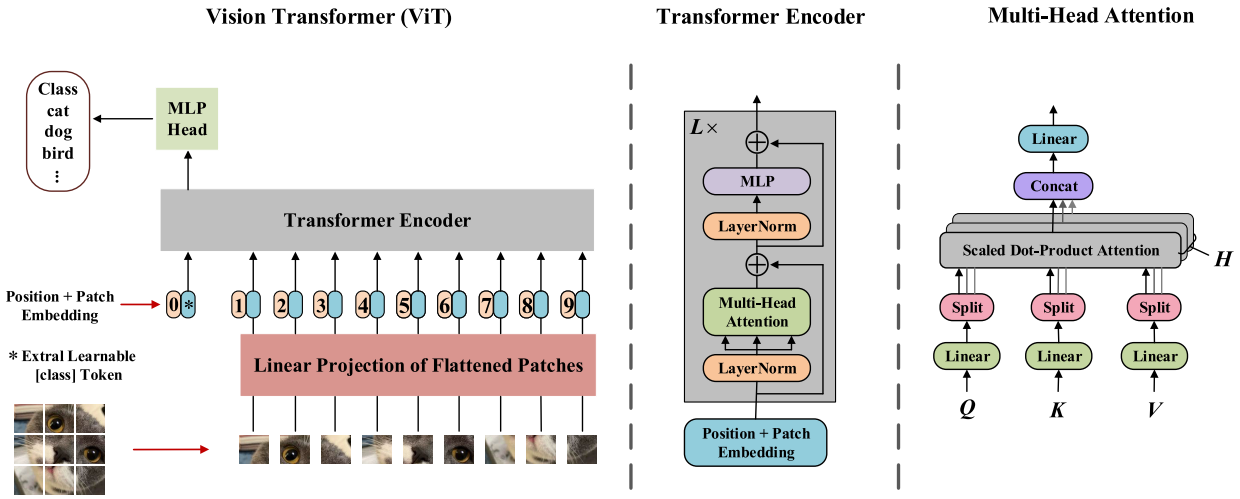


Fig. 2. ViT for natural image classification.

Before each block, layernorm (LN) is applied to normalize the embedded features at each position. What ViT wants is the best representation of the learnable class token. From the operating principle of the Transformer encoder, the process of getting the optimal representation is like looking up a dictionary created by it. However, in fact, the encoder does not have a specific token to represent the keys; it has a vectorized representation of these concepts. Therefore, the keys used for searching (called queries, $\boldsymbol{Q}$) do not exactly match any keys ($\boldsymbol{K}$) in the dictionary. The solution is to calculate the similarity between the query and each key in the dictionary and then use the softmax function to convert these similarity scores into weights that sum to 1. The model can then calculate a weighted sum of the corresponding values ($\boldsymbol{V}$)

$$\text{Attention}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}}{\sqrt{d}}\right)\boldsymbol{V} \tag{10}$$

where $\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V} \in \mathcal{R}^{(N+1)\times d}$ represent queries, keys, and values, respectively; d represents the dimension of each query, each key, and each value; $\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}} \in \mathcal{R}^{(N+1)\times(N+1)}$ represents the similarity score of each query/key pair; and $(1/\sqrt{d}$ is a scaling factor. The final output shape is $[N+1, d]$, which means that each query has one row, and each row represents the query result.

The optimal representation of a position is a linear combination of representations of several related positions. In addition, the representation of each patch encodes many different features; if only a single attention layer is used, all the features can only be queried at once. MSA allows the model to project many different patch features into different subspaces, each of which focuses on a subset of the patch features.

The small MLP at the top of the encoder contains two linear layers using the Gaussian error linear unit (GELU) as the activation function. The expression of the GELU activation function is given as follows:

$$\text{GELU}(x) = x\Phi(x) = \frac{x}{\sqrt{2\pi}}\int_{-\infty}^{x} e^{-t^2/2}dt \tag{11}$$

where $\Phi(x)$ is the cumulative distribution function for the standard Gaussian distribution. Finally, the whole calculation process of ViT can be summarized by the following formulas:

$$\boldsymbol{z}_0 = \left(\boldsymbol{x}_{\text{class}}, \boldsymbol{x}_p^1\boldsymbol{W}, \boldsymbol{x}_p^2\boldsymbol{W}, \cdots, \boldsymbol{x}_p^N\boldsymbol{W}\right)^{\mathrm{T}} + \boldsymbol{W}_{\text{pos}} \tag{12}$$

$$\boldsymbol{z}_l' = \text{MSA}(\text{LN}(\boldsymbol{z}_{l-1})) + \boldsymbol{z}_{l-1} \tag{13}$$

$$\boldsymbol{z}_l = \text{MLP}\left(\text{LN}\left(\boldsymbol{z}_l'\right)\right) + \boldsymbol{z}_l' \tag{14}$$

$$\boldsymbol{y} = \text{MLP}\left(\text{LN}\left(\boldsymbol{z}_L^0\right)\right) \tag{15}$$

where $\boldsymbol{W} \in \mathcal{R}^{3\cdot P^2\times D}$ denotes the weight of the linear projection layer used to get the underlying features of the patches, D is the projection dimension, $\boldsymbol{x}_{\text{class}} \in \mathcal{R}^D$ denotes the learnable class token, $\boldsymbol{W}_{\text{pos}} \in \mathcal{R}^{(N+1)\times D}$ stands for the learnable position embeddings, L means that there are a total of L encoders stacked, $l = 1, 2, 3, \ldots, L$ indicates the encoder number, $\boldsymbol{z}_l$ represents the output features of the lth encoder, $\boldsymbol{z}_L^0$ represents the best class token representation learned by the top-level encoder, and $\boldsymbol{y}$ represents the final prediction probability and is used to calculate the cross-entropy to train the entire model.

D. GLOM System

A fundamental disadvantage of CapsNets is that they use a set of preallocated capsules to model a set of possible parts. This makes it difficult to judge whether the car headlights and eyes are really different parts. If they are modeled by the same capsule, the capsule cannot predict the identity of the whole. If they are modeled by different capsules, the similarity in their relationship with respect to their whole cannot be captured. Based on the above analysis, Hinton proposed an imaginary system called GLOM to represent the part-whole hierarchies in neural networks. This system simultaneously learns the embedding of the patch at the lowest level, the subpart level, the part level, the object level, and the scene level by preassigning a ubiquitous multilevel universal capsule at the location of each image patch. This ubiquitous multilevel universal capsule can be likened to a gene sequence. Each cell (patch here) has a different expression intensity for each gene. The GLOM system draws on the above ideas and uses the islands composed of identical vectors at various levels to represent a node in a parse tree. Fig. 3 shows the basic diagram of this system.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

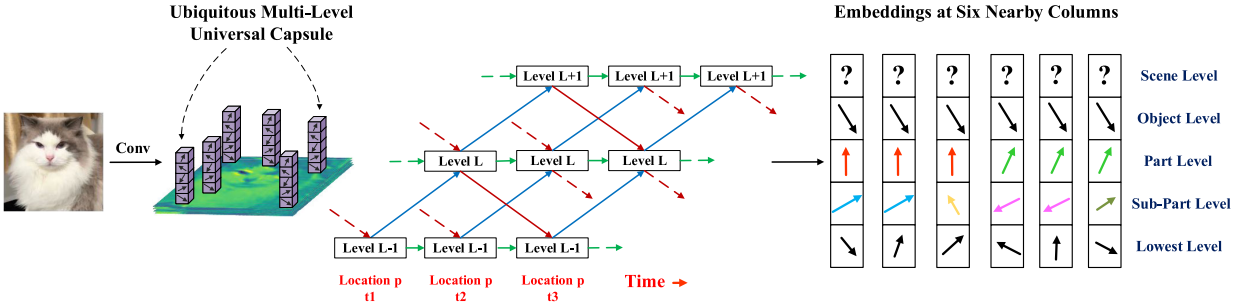


Fig. 3. Overview of the GLOM system.

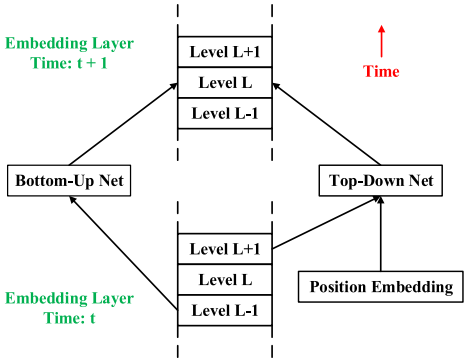


Fig. 4. Top-down and bottom-up predictions between two adjacent levels of GLOM for a single column.

The core component of the GLOM system is a multilevel embedding learning module based on time steps. For a static image with only one prominent object,[1] the embeddings at a level should gradually stabilize over time to produce different islands of nearly identical vectors. With the increase in the embedding level, these islands should gradually become larger to form a node in the parse tree. The root node of the parse tree corresponds to the object-level embedding, and all of the locations (belonging to the same object) should learn exactly the same object-level vector. In addition, the top-level embedding, i.e., scene-level embedding, contains the 3-D pose information of the object relative to the camera.

In GLOM, the multilevel embeddings of one patch form a column. For each column, at each discrete time t, the embedding at a level is updated by the average of four contributions. Three of them are shown in Fig. 3: 1) the prediction obtained by the top-down neural network that takes the upper level embedding as input at the previous time (red arrow); 2) the prediction obtained by the bottom-up neural network taking the lower level embedding as input at the previous time (blue arrow); and 3) the embedding at the same level at the previous time (green arrow). The top-down and bottom-up networks are two different networks, and they are shared across locations and times. However, they can differ between pairs of levels. Fig. 4 shows the working principle diagram of these two networks. Among them, the bottom-up network simply predicts advanced embedding. However, the top-down network predicts low-level embeddings, which is related to the absolute or relative position of the patch, so the information about the location should be added.

[1]GLOM could also parse the part-whole hierarchies in video streams and static images with multiple objects.

In addition to the three contributions shown in Fig. 3, for each discrete time t and each column, the update of the embedding at each level is also affected by the embeddings at the same level in nearby columns. For each level, the interactions between the embeddings in different columns are implemented through an attention-weighted local smoother. Let $\boldsymbol{E} = (\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_N)^{\mathrm{T}} \in \mathcal{R}^{N\times d}$ represent the embeddings at the same level in different columns, where N denotes the total number of columns (patches) and d denotes the dimension of each embedding. The interactions between these embeddings can be expressed as

$$\text{Interact}(\boldsymbol{E}) = \text{softmax}\left\{\frac{\boldsymbol{E}[\text{norm}(\boldsymbol{E})]^{\mathrm{T}}}{\sqrt{d}}\right\}\boldsymbol{E} \tag{16}$$

where "norm" represents the vector normalization operation. We can see that the interactions between the embeddings at the same level in different columns actually calculate an attention-weighted average, which borrows from the scaled dot-product attention in Transformer. Therefore, the fourth contribution mentioned above can be simply summarized as the attention-weighted average of the embeddings at the same level in different columns at the previous time.

III. Proposed Framework

HIC based on AL under a few training samples is the baseline of this study. We develop a CAL framework, which includes two novel and excellent base classifiers and a CAL scheme that considers both the diversity of actively selected samples and the budget of expert annotation. The first base classifier is the CapsNet combined with ViT, which is named CapsViT, and the second is improved on the GLOM system, which is named CapsGLOM. Next, we cover the components of the proposed framework in detail.

A. CapsViT

CapsNet can not only detect whether a specific feature appears but also accurately locate the position of the feature in the unit hypersphere embedding space. However, CapsNet does not take into account the global correlation between PrimaryCaps. Learning the optimal "primary" capsules[2] by exploring the global correlation between the PrimaryCaps at each location is our motivation for introducing ViT into the CapsNet. In Fig. 5, we take the Kennedy Space Center (KSC) dataset as an example to show the complete diagram of the proposed CapsViT.

[2]The PrimaryCaps here refer to the deeper PrimaryCaps, which has been refined by ViT.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

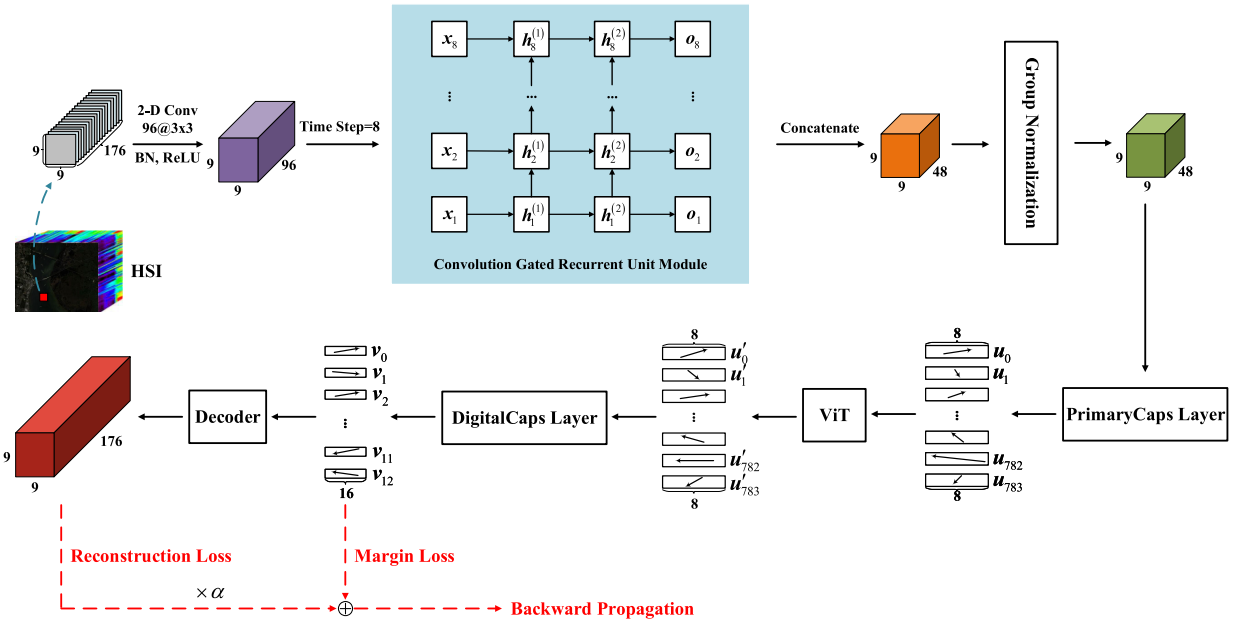


Fig. 5. Proposed CapsViT for HIC. The actual training process is carried out in minibatches; for the sake of demonstration, only one sample is taken here as an example.

CapsViT is an end-to-end model that does not require dimension reduction preprocessing. As shown in Fig. 5, to make full use of spectral characteristics, at the beginning of CapsViT, we use a convolution layer and a convolution-gated recurrent unit (ConvGRU) module with two hidden layers to learn local and nonlocal cross-channel spectral dependencies. Taking the KSC dataset (there are 176 spectral bands) as an example, suppose that the spatial size of the training sample $\boldsymbol{x}$ is 9×9, i.e., $\boldsymbol{x} \in \mathcal{R}^{176\times 9\times 9}$. A minibatch of training samples $\boldsymbol{x}_{\text{batch}} \in \mathcal{R}^{\text{bs}\times 176\times 9\times 9}$ (where bs denotes the batch size) is sent to the first convolution layer

$$\boldsymbol{f}_0 = \text{ReLU}(\text{BN}(\boldsymbol{W}_0 * \boldsymbol{x}_{\text{batch}} + \boldsymbol{b}_0)) \tag{17}$$

where $*$ represents the 2-D convolution operation, $\boldsymbol{W}_0$ and $\boldsymbol{b}_0$ are weights and biases, respectively, and BN represents the batch normalization. There are 96 convolutional kernels in this layer, and the size of the receptive field (RF) is 3×3.

The output of the first convolution layer $\boldsymbol{f}_0 \in \mathcal{R}^{\text{bs}\times 96\times 9\times 9}$ is then evenly divided into eight parts along the channel dimension and sent to the ConvGRU module as input at each time step to learn local and nonlocal cross-channel spectral dependencies. The ConvGRU module is composed of two layers of ConvGRU, and the structure of the ConvGRU is shown in Fig. 6. For the first layer's ConvGRU, there are 12 convolutional kernels in each gate controller. For the second layer's ConvGRU, there are six convolutional kernels in each gate controller. Let $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_8 \in \mathcal{R}^{\text{bs}\times 12\times 9\times 9}$ represent the input at each time step, respectively; then, the output of the first layer's ConvGRU at each time step can be expressed by the following formulas:

$$\begin{aligned}
\boldsymbol{z}_t &= \sigma(\boldsymbol{W}_{xz} * \boldsymbol{x}_t + \boldsymbol{W}_{hz} * \boldsymbol{h}_{t-1}) \\
\boldsymbol{r}_t &= \sigma(\boldsymbol{W}_{xr} * \boldsymbol{x}_t + \boldsymbol{W}_{hr} * \boldsymbol{h}_{t-1}) \\
\boldsymbol{g}_t &= \tanh\big(\boldsymbol{W}_{xg} * \boldsymbol{x}_t + \boldsymbol{W}_{hg} * (\boldsymbol{r}_t \otimes \boldsymbol{h}_{t-1})\big) \\
\boldsymbol{y}_t &= \boldsymbol{h}_t = \boldsymbol{z}_t \otimes \boldsymbol{h}_{t-1} + (1 - \boldsymbol{z}_t) \otimes \boldsymbol{g}_t
\end{aligned} \tag{18}$$

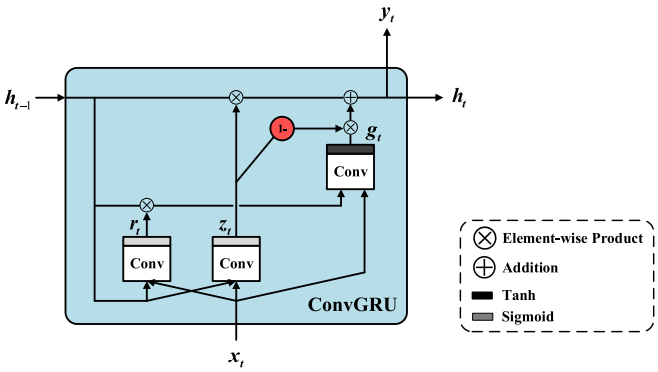


Fig. 6. Architecture of ConvGRU.

where $\boldsymbol{W}_{xz}$, $\boldsymbol{W}_{xr}$, and $\boldsymbol{W}_{xg}$ are the convolutional kernels in each gated unit for the input $\boldsymbol{x}_t$; $\boldsymbol{W}_{hz}$, $\boldsymbol{W}_{hr}$, and $\boldsymbol{W}_{hg}$ are the convolutional kernels for the state $\boldsymbol{h}_t$; $\otimes$ represents elementwise multiplication; σ represents the sigmoid activation function; and tanh denotes the Tanh activation function. The output of the first layer's ConvGRU at each time step $\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_8 \in \mathcal{R}^{\text{bs}\times 12\times 9\times 9}$ is used as the input of the second layer's ConvGRU. Finally, we end up with eight outputs at eight time steps $\boldsymbol{o}_1, \boldsymbol{o}_2, \ldots, \boldsymbol{o}_8 \in \mathcal{R}^{\text{bs}\times 6\times 9\times 9}$. Although $\boldsymbol{o}_8$ integrates the information of $\boldsymbol{o}_1, \boldsymbol{o}_2, \ldots, \boldsymbol{o}_7$, we concatenate them to make full use of shallow and deep spectral dependencies. Then, in order to make the learning process fast and smooth, group normalization with a grouping parameter of 8 (corresponding to eight time steps) is used.

Next, the obtained features are sent to the PrimaryCaps layer, in which there are 128 convolutional kernels having RF size 3×3. To adjust the number of PrimaryCaps obtained, the convolution without padding is applied in this layer. By integrating eight neurons into one capsule along the channel dimension, we can obtain 784 ($16 \times 7 \times 7$) PrimaryCaps $\boldsymbol{u}_0, \boldsymbol{u}_1, \ldots, \boldsymbol{u}_{783}$. The mold of a PrimaryCap represents the probability that a certain entity feature (such as a cat's eye)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETER CONFIGURATION OF CAPSVIT, AND THE INPUT AND OUTPUT SHAPES OF EACH LAYER

Layer (type)	Kernel Number	Kernel Shape	Stride	Padding	Input Shape	Output Shape
Conv2d-1	96	3×3	1	1	[-1, 176, 9, 9]	[-1, 96, 9, 9]
ConvGRU-2	12	3×3	1	1	[-1, 12, 9, 9]	[-1, 12, 9, 9]
ConvGRU-3	6	3×3	1	1	[-1, 12, 9, 9]	[-1, 6, 9, 9]
GN-4	–	–	–	–	[-1, 48, 9, 9]	[-1, 48, 9, 9]
PrimaryCaps-5	128	3×3	1	0	[-1, 48, 9, 9]	[-1, 784, 8]
ViT-6	–	–	–	–	[-1, 784, 8]	[-1, 784, 8]
DigitalCaps-7	–	–	–	–	[-1, 784, 8]	[-1, 13, 16]
FC-8	–	–	–	–	[-1, 16]	[-1, 128]
ConvTrans2d-9	256	3×3	1	0	[-1, 128, 1, 1]	[-1, 256, 3, 3]
ConvTrans2d-10	256	3×3	2	1	[-1, 256, 3, 3]	[-1, 256, 5, 5]
ConvTrans2d-11	176	3×3	2	1	[-1, 256, 5, 5]	[-1, 176, 9, 9]

appears, and its direction represents postural information about the entity (such as the color or size of a cat's eye). Each entity feature appears within a specific region of the unit hypersphere. Therefore, we next feed these PrimaryCaps into ViT to learn the optimal representation of the PrimaryCaps at each "location," which takes into account the global correlation between all the PrimaryCaps. We represent the ViT-treated PrimaryCaps as $\boldsymbol{u}'_0, \boldsymbol{u}'_1, \ldots, \boldsymbol{u}'_{783}$; they are more discriminative and, therefore, have a positive impact on classification. It should be added that, in the ViT module, the mapping dimension of the queries, keys, and values is 16, and both linear layers in the MLP at the top of the encoder have an output dimension of 8.[3] In addition, there are five encoders stacked, and each encoder uses four-head attention.

Finally, there are 13 DigitalCaps $\boldsymbol{v}_0, \boldsymbol{v}_1, \ldots, \boldsymbol{v}_{12}$ in the DigitalCaps layer, corresponding to 13 types of ground objects in the KSC dataset. The DigitalCaps corresponding to the training samples' labels are input to the decoder to reconstruct the input samples, and the decoder consists of an FC layer and three transposed convolution layers. During BP, (9) is used as the loss function to train the model. To facilitate understanding, the parameter configuration of the model and the input and output shapes of each layer are listed in Table I (Conv2d-1 stands for the first convolution layer and ConvTrans stands for the transposed convolution).

B. CapsGLOM

The GLOM system is an imaginary and powerful system. However, some of the ideas for building this system cannot make it run perfectly, which is mainly reflected in the following two aspects.

1) All positions belonging to the same object should be given exactly the same object-level embedding, but, in practice, it is difficult to tell which locations belong to the same object.
2) If we regard the ubiquitous multilevel universal capsule as a gene sequence, the gene sequences belonging to different objects should be different.

In view of the above two deficiencies, we develop a new deep model for HIC, namely, CapsGLOM. In CapsGLOM, we learn different multilevel embeddings at different locations and then use the GLOM system's multilevel embedding learning module to learn the consensus opinions (i.e., islands of nearly identical vectors) at each level. Fig. 7 shows the framework of the proposed CapsGLOM.

As shown in Fig. 7, the first half of CapsGLOM is the same as CapsViT; that is, we use a two-layer ConvGRU module to learn local and nonlocal cross-channel spectral dependencies, followed by group normalization. Let $\boldsymbol{f}_1 \in \mathcal{R}^{\text{bs}\times 48\times 9\times 9}$ represent the output of group normalization. Next, there are two branches, the first of which uses an FC layer to learn the underlying features at each location that serve as inputs to the bottom-up network to predict the lowest level embeddings. To be clear, locations could be as fine-grained as pixels, or they could correspond to larger image patches. In CapsGLOM, we define location as the location of each pixel in the feature maps. The acquisition of the underlying features at each location can be expressed by the following formula:

$$\boldsymbol{f}_{\text{under}} = \text{GELU}(\text{LN}(\text{Reshape}(\boldsymbol{f}_1)\boldsymbol{W}_1 + \boldsymbol{b}_1)) \tag{19}$$

where $\text{Reshape}(\boldsymbol{f}_1) \in \mathcal{R}^{\text{bs}\times 81\times 48}$ represents the reshaping operation on $\boldsymbol{f}_1$, $\boldsymbol{W}_1 \in \mathcal{R}^{48\times 32}$ and $\boldsymbol{b}_1 \in \mathcal{R}^{32}$ represent the weights and biases of the FC layer, respectively, and $\boldsymbol{f}_{\text{under}} \in \mathcal{R}^{\text{bs}\times 81\times 32}$ stands for the learned underlying features with dimension 32. The second branch uses 2-D convolution to learn five-level embeddings $\boldsymbol{f}_{\text{lowest}}, \boldsymbol{f}_{\text{sub-part}}, \boldsymbol{f}_{\text{part}}, \boldsymbol{f}_{\text{object}}$, and $\boldsymbol{f}_{\text{scene}}$ at each location. Since we define each "pixel" as a location, the pixelwise convolution, i.e., 1×1 convolution, is used here. The multilevel embeddings at each location can be calculated by the following formula:

$$\begin{aligned}(\boldsymbol{f}_{\text{scene}}, \boldsymbol{f}_{\text{object}}, \boldsymbol{f}_{\text{part}}, \boldsymbol{f}_{\text{sub-part}}, \boldsymbol{f}_{\text{lowest}})^{\text{T}} \\ = \text{LN}(\text{Reshape}(\text{GELU}(\boldsymbol{W}_2 * \boldsymbol{f}_1 + \boldsymbol{b}_2)))\end{aligned} \tag{20}$$

where $\boldsymbol{W}_2$ and $\boldsymbol{b}_2$ denote the weights and biases of the pixelwise convolution, respectively. Because each position has five levels of embedding and each level of embedding has a dimension of 32, the number of kernels of pixelwise convolution is 160. The feature maps obtained by pixelwise convolution are divided into five parts along the channel dimension on average and then reshaped into multilevel embeddings of all locations. It is important to note that, although pixelwise convolution and FC layer are equivalent, the order of GELU activation functions and LN operations in the two branches above is not the same. In other words, you could change the order of GELU, Reshape, and LN in the second branch to Reshape, LN, and GELU so that an FC layer or a pixelwise convolution can

[3]The second linear layer in the MLP is used to restore the original dimension.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

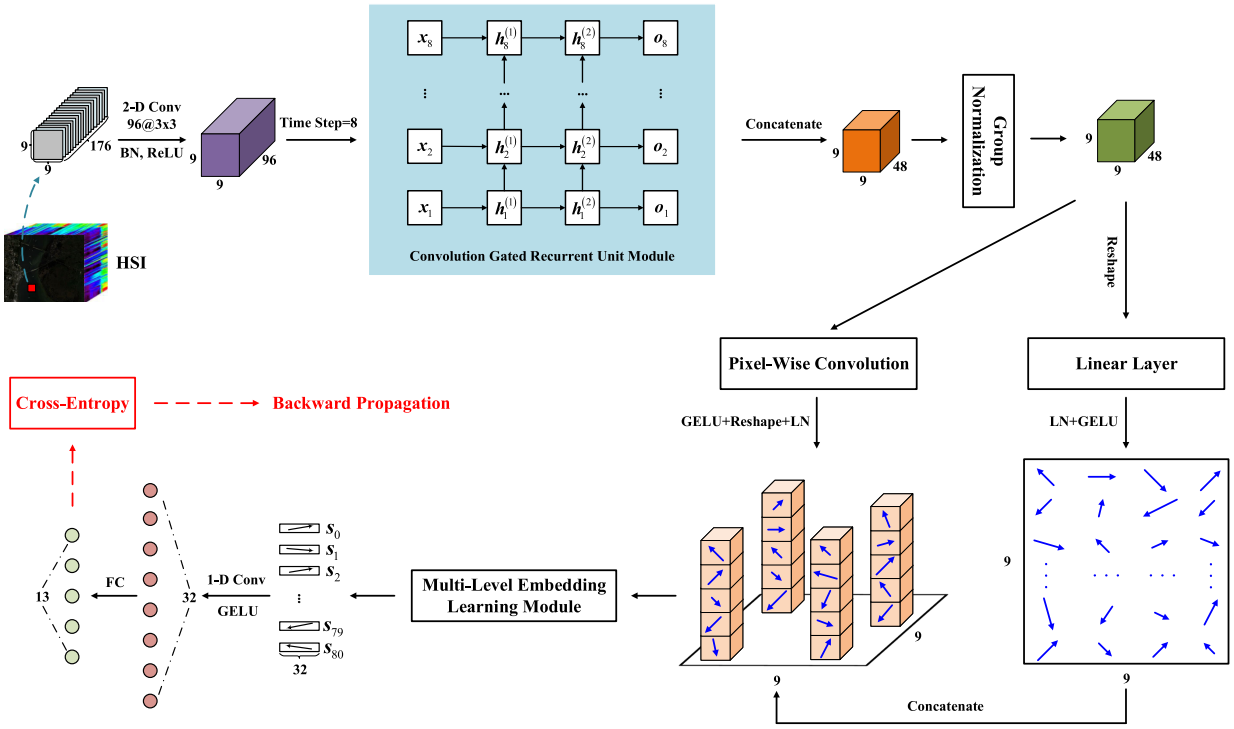


Fig. 7. Proposed CapsGLOM for HIC. The actual training process is carried out in minibatches; for the sake of demonstration, only one sample is taken here as an example.

be used to simultaneously learn the underlying features and multilevel embeddings at each location. However, the way we build the multilevel embeddings borrows from the way CapsNet builds the PrimaryCaps,[4] and of course, you can also build the multilevel embeddings using stride convolution with padding instead of pixelwise convolution.

Next, the underlying features and multilevel embeddings of each location are fed into the GLOM system's multilevel embedding learning module to learn the consensus opinions at each level. In the multilevel embedding learning module, the setting of the time step will affect the learning speed and performance of the model. In the process of designing the model, many experiments show that three time steps are the best. In addition, both the bottom-up and top-down networks are composed of two linear layers activated by GELU, and the output dimensions of these two linear layers are 128 and 32, respectively. The embeddings at each level predicted by these two networks can be expressed by the following formulas:

$$(\boldsymbol{f}_{\text{scene}},\boldsymbol{f}_{\text{object}},\boldsymbol{f}_{\text{part}},\boldsymbol{f}_{\text{sub-part}},\boldsymbol{f}_{\text{lowest}})^{\text{T}} \leftarrow \text{bottom-up}(\boldsymbol{f}_{\text{object}},\boldsymbol{f}_{\text{part}},\boldsymbol{f}_{\text{sub-part}},\boldsymbol{f}_{\text{lowest}},\boldsymbol{f}_{\text{under}}) \tag{21}$$

$$(\boldsymbol{f}_{\text{object}},\boldsymbol{f}_{\text{part}},\boldsymbol{f}_{\text{sub-part}},\boldsymbol{f}_{\text{lowest}})^{\text{T}} \leftarrow \text{top-down}(\boldsymbol{f}_{\text{scene}},\boldsymbol{f}_{\text{object}},\boldsymbol{f}_{\text{part}},\boldsymbol{f}_{\text{sub-part}}). \tag{22}$$

Let $\boldsymbol{E}_t = (\boldsymbol{f}_{\text{scene}},\boldsymbol{f}_{\text{object}},\boldsymbol{f}_{\text{part}},\boldsymbol{f}_{\text{sub-part}},\boldsymbol{f}_{\text{lowest}})^{\text{T}}$ denote the multilevel embeddings of all locations at time t; then, at the next time $t+1$, it can be updated by the following formula:

$$\boldsymbol{E}_{t+1} = \big[\text{bottom-up}(\boldsymbol{E}_t,\boldsymbol{f}_{\text{under}}) + \text{top-down}(\boldsymbol{E}_t) + \boldsymbol{E}_t + \text{Interact}(\boldsymbol{E}_t)\big]/(3,4,4,4,4)^{\text{T}} \tag{23}$$

where Interact represents the interactions between the embedding vectors at the same level in different columns; please refer to (16).

Since we are performing a classification task, and each position (pixel) may represent a background pixel or some other type of ground object, we use the scene-level embeddings at each position to perform the classification as these embeddings characterize the sample's information that contains the entire scene. They are fed into a 1-D convolution layer to learn the integrated features for the final classification. Finally, cross-entropy is used as the loss function to train the entire model. To facilitate understanding, the parameter configuration of the model, and the input and output shapes of each layer are listed in Table II (Conv2d-6 stands for pixelwise convolution).

C. CAL Scheme

At present, most uncertainty-based deep AL methods for HIC only consider the uncertainty of the selected samples and do not take into account the diversity and the budget of expert annotation. In this section, we present a CAL scheme that considers both the uncertainty and diversity of actively selected samples, as well as the cost of expert annotation.

Our proposed CAL scheme belongs to an AL method based on uncertainty. BvSB and EP are two commonly used uncertainty measures. For any unlabeled sample $\boldsymbol{x}_i$, we can use a trained model to estimate its class membership probabilities. Let $y_i = (y_{i1}, y_{i2}, \ldots, y_{iC})^{\text{T}} \in \mathcal{R}^C$ denote the estimated

[4]The multilevel embeddings of all locations in CapsGLOM are not squashed because their molds do not indicate the probability of occurrence of an entity feature.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PARAMETER CONFIGURATION OF CAPSGLOM, AND THE INPUT AND OUTPUT SHAPES OF EACH LAYER

Layer (type)	Kernel Number	Kernel Shape	Stride	Padding	Input Shape	Output Shape
Conv2d-1	96	3×3	1	1	[-1, 176, 9, 9]	[-1, 96, 9, 9]
ConvGRU-2	12	3×3	1	1	[-1, 12, 9, 9]	[-1, 12, 9, 9]
ConvGRU-3	6	3×3	1	1	[-1, 12, 9, 9]	[-1, 6, 9, 9]
GN-4	–	–	–	–	[-1, 48, 9, 9]	[-1, 48, 9, 9]
Linear-5	–	–	–	–	[-1, 81, 48]	[-1, 81, 32]
Conv2d-6	160	1×1	1	0	[-1, 48, 9, 9]	[-1, 160, 9, 9]
Bottom-up-7	–	–	–	–	[-1, 81, 5, 32]	[-1, 81, 5, 32]
Top-down-8	–	–	–	–	[-1, 81, 4, 32]	[-1, 81, 4, 32]
Conv1d-9	1	81	1	0	[-1, 81, 32]	[-1, 1, 32]
FC-10	–	–	–	–	[-1, 32]	[-1, 13]

posterior probability; then, the BvSB-based uncertainty of the unlabeled sample x_i can be expressed as

$$\mathrm{BvSB}(\boldsymbol{x}_i) = P_B(y_i) - P_{\mathrm{SB}}(y_i) \tag{24}$$

where $P_B(y_i)$ represents the highest (probability) value in y_i and $P_{\mathrm{SB}}(y_i)$ represents the second highest value. For this criterion, the smaller the BvSB value of a sample, the fuzzier the classifier will judge the category of the sample. Entropy is usually used to describe the degree of confoundedness, so it can also be used as an uncertainty measure in AL, which can be calculated by the following formula:

$$H(\boldsymbol{x}_i) = -\sum_{c=1}^{C} P(y_{ic}) \log P(y_{ic}) \tag{25}$$

where log represents the natural logarithm. For any unlabeled sample, the higher the entropy of the estimated posterior probabilities, the more difficult it is for the classifier to judge its category. For the above two uncertainty measures, BvSB is usually better than prediction EP [2], and thus, we adopt the BvSB-based query strategy in our proposed CAL scheme.

In the proposed CAL scheme, CapsViT is used to calculate the uncertainty of candidate samples, and CapsGLOM is used to assist expert annotation and ensure the diversity of actively selected samples. The main advantage of CAL is that it can make full use of the knowledge learned from the two base classifiers and ensure the diversity of actively selected samples without designing any diversity criteria. Most importantly, compared with the traditional batch-based AL, CAL fully considers the cost of expert labeling. Let X^L represent the labeled sample set and X^C represent the candidate sample set. It should be added that, for datasets with a small number of samples, the whole test set can be used as the candidate set. If the number of samples is too large, in order to reduce the computing time and memory requirements, an appropriate number of unlabeled samples are generally selected to construct the candidate set (pool). Back to the point, in each AL iteration, we first train CapsViT and CapsGLOM with X^L, then send X^C to CapsViT to calculate the BvSB value of each candidate sample, and send it to CapsGLOM for classification. Suppose that there are M candidate samples, that is, $X^C = \{x_1^C, x_2^C, \ldots, x_M^C\}$, and we can get the indices that sort the BvSB values $\{\mathrm{BvSB}(x_1^C), \mathrm{BvSB}(x_2^C), \ldots, \mathrm{BvSB}(x_M^C)\}$ in the ascending order

$$\mathrm{indices} = \mathrm{argsort}\left(\mathrm{BvSB}\left(X^C\right)\right). \tag{26}$$

Next, we create a dictionary for CapsGLOM with the keys $0, 1, \ldots, C-1$, and each key corresponds to a collector (an empty list). Let us represent the indices as {ind_0, ind_1, …, ind_(M − 1)}, and then, the candidate samples corresponding to the indices can be represented as $\{X^C[\mathrm{ind_0}], X^C[\mathrm{ind_1}], \ldots, X^C[\mathrm{ind_(M-1)}]\}$. The above-reordered candidate samples are, in turn, added to the collector with the category estimated by CapsGLOM as the key. In this way, the BvSB values of the samples in each collector remain in the order from smallest to largest. Finally, we select only the first sample in each collector for expert annotation. If the expert has difficulty labeling the first sample in a particular collector, he can try labeling the second sample and so on. Accordingly, the proposed CAL scheme not only considers the uncertainty and diversity of actively selected samples but also takes into account the cost of expert labeling. As a conclusion, we present the algorithm flow of our proposed CAL framework in Algorithm 1.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

To analyze the effectiveness and superiority of our proposed CAL framework, we conducted extensive experiments on four benchmark HSI datasets: the KSC dataset, the Houston University 2013 (HU2013) dataset, the Pavia University (PU) dataset, and the Chikusei dataset. We run two experiments on each dataset: an AL experiment with few training samples and a supervised classification experiment with sufficient samples. In the AL experiments, we compare CapsViT, CapsGLOM, and the slightly modified A2S2K-RN [10]. In the supervised classification experiments, we compare the proposed CapsViT and CapsGLOM with some recently proposed deep models: HybridSN [4], CAEVT [6], A2S2K-RN [10], CapsNet [11], SpectralNET [13], and SpectralFormer [20]. All the experiments use three metrics to evaluate the performance of each model: overall accuracy (OA), average accuracy (AA), and kappa coefficient (κ). Finally, all the experiments were performed on a system with an NVIDIA GeForce RTX 3090 GPU accelerator (with 24-GB GPU memory).

Some of the hyperparameters used to build CapsViT and CapsGLOM that are given in Tables I and II, respectively. Before starting to introduce the experimental part, we present some other hyperparameter settings for these two models in Table III, including initial learning rate (η_0), learning rate scheduling, optimizer, weight decay coefficient, and training epoch. The training epoch here refers to the default value,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

TABLE III
SOME OTHER HYPERPARAMETER SETTINGS FOR CAPSVIT AND CAPSGLOM

Model	Initial Learning Rate (η_0)	Learning Rate Scheduling	Optimizer	Weight Decay	Training Epoch
CapsViT	5e-4	$\eta_0 \times 0.99^{\text{epoch}}$	Adam	1e-3	200
CapsGLOM	1e-3	$\eta_0 \times 0.99^{\text{epoch}}$	Adam	0.00	200

Algorithm 1 CAL Framework

Input: Labeled sample set: X^L; Candidate sample set: X^C; The number of AL iterations: I; The number of categories: C.
Output: Trained CapsViT and CapsGLOM.

Initialize $i = 0$;
while $i < I$ **do**
  Train CapsViT and CapsGLOM with X^L;
  Calculate the BvSB values of X^C via CapsViT: candidate_bvsb=BvSB(X^C);
  Classify X^C via CapsGLOM: pred_capsglom=CapsGLOM(X^C);
  Get the indicies: indices = argsort(candidate_bvsb);
  dict = {0: [ ], 1: [ ], . . . , C-1: [ ]};
  Collet the most informative samples in the current iteration: X^I=[ ];
  M = len(X^C);
  for $j = 0$ **to** $(M - 1)$ **do**
    ind=indices[j];
    pred_label=pred_capsglom[ind];
    dict[pred_label].append$(X^C$[ind]$)$;
  end
  for $c = 0$ **to** $(C - 1)$ **do**
    Select only the first sample in each collector: x_c=dict[c][0];
    Ask the expert to annotate x_c;
    while *there's a difficulty here* **do**
      Ask the expert to annotate the next sample in the current collector: dict[c];
    end
    X^I.append(x_c);
  end
  Update the labeled sample set: $X^L \leftarrow X^L \cup X^I$;
  Update the candidate sample set: $X^C \leftarrow X^C - X^I$;
  $i \leftarrow i + 1$;
end
Train CapsViT and CapsGLOM with X^L for the last time.

which should be adjusted in practice according to the characteristics of the dataset and the number of initial training samples.

A. Experiment 1: The Kennedy Space Center Dataset

The KSC dataset was captured in 1996 by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) at the KSC in Florida. AVIRIS collected 224 bands of 10 nm width, with a central wavelength of 400–2500 nm. The spatial resolution of the KSC dataset is 18 m; after removing the water absorption and low signal-to-noise ratio (SNR) bands, 176 bands are used for analysis. This dataset has a spatial dimension of 512 × 614 pixels and contains a total of 5211 samples in 13 categories. Fig. 8(a) shows the three-band (bands 29, 19, and 9) false-color image of this dataset, and Fig. 8(b) shows the color-coded ground truth.

For the AL experiment on this dataset, only one sample from each category was randomly selected as the initial training sample, that is to say, the initial training set consists of 13 training samples. Fig. 8(c) shows the training samples selected, and the spatial size of each sample is 9 × 9. We will discuss the effect of different spatial sizes of training samples on classification results based on this dataset. However, considering that a large spatial size usually means a larger amount of computation, experiments on the other datasets all adopt the spatial size of 9 × 9. We set the number of AL iterations to 4, which means that a total of 52 additional samples were selected for expert labeling. In addition, we made a small modification to A2S2K-RN when comparing it. The original A2S2K-RN uses convolution kernels of 1 × 1 and 7 × 7 to extract spectral and spatial features, respectively; we replace these two convolution kernels with three convolution kernels of 1 × 1, 3 × 3, and 5 × 5. To prevent batch normalization from encountering the situation that a minibatch only contains one training sample, the batch size used by all the compared models for training was set to 30. Finally, one thing to be clear about, in order to reproduce our experiments, we used a random seed (seed = 0) to select the initial training samples, and in order to eliminate the influence of parameter initialization on the experimental results, we also used this random seed to control the initialization of each model.[5]

In addition to the 13 training samples in the initial training set, the remaining 5198 samples were used as the test samples and candidate samples used for active selection. It should be noted that, because the total number of samples in the KSC dataset is not large, we did not separately divide the candidate sample set. In Table IV, we report the classification results achieved by A2S2K-RN, CapsViT, and CapsGLOM on the initial training set and the training sets obtained after two and four AL iterations. In this experiment, A2S2K-RN was also trained for 200 epochs, and we assume that the expert would be able to annotate the first unmarked sample in each collector.

As shown in Table IV, when there is only one training sample for each category, the performance of each model on the test set is unsatisfactory. Besides that, there are great differences in the ability of each model to identify each category. However, it can be found that, for class 4 (Pine), which is difficult for all three models to correctly identify,

[5]All the compared models are implemented by the PyTorch DL framework, and we set up the following commands: torch.manual_seed(0), torch.cuda.manual_seed(0), torch.backends.cudnn.deterministic = True, and torch.backends.cudnn.benchmark = False.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

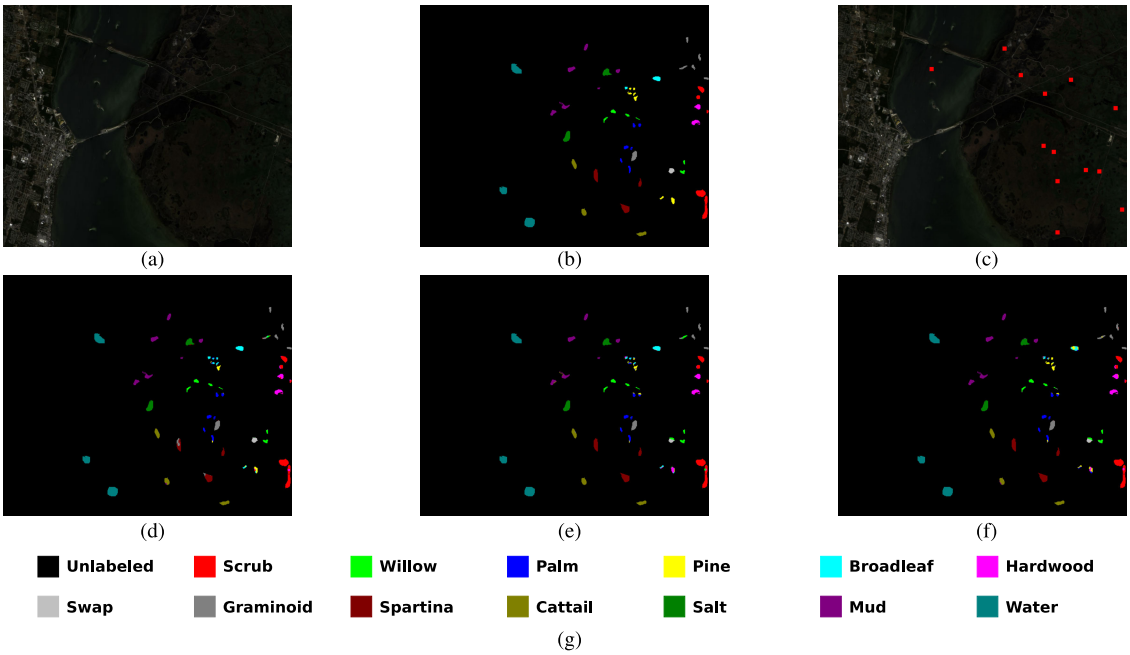


Fig. 8. Visualization of the classification maps obtained by different models on the KSC dataset. (a) Three-band false-color composite. (b) Ground truth. (c) Training samples. (d)–(f) Classification maps obtained by A2S2K-RN, CapsViT, and CapsGLOM after four AL iterations, respectively. (g) Legend.

TABLE IV
AL EXPERIMENTS ON THE KSC DATASET USING 13 INITIAL TRAINING SAMPLES (THE SPATIAL SIZE OF TRAINING SAMPLES IS 9×9)

Class		Active Learning Iteration=0			Active Learning Iteration=2			Active Learning Iteration=4		
		A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM
Accuracies(%)	C1	96.18	98.82	**99.61**	**97.89**	93.55	97.63	95.13	**96.71**	94.87
	C2	90.50	**90.91**	26.45	54.96	**71.49**	39.26	99.59	**100.00**	95.04
	C3	**79.61**	69.41	65.10	**99.61**	96.08	94.90	**95.69**	88.24	88.63
	C4	14.34	7.57	**18.73**	16.73	17.53	**29.48**	48.61	33.47	**69.72**
	C5	**71.25**	20.00	57.50	**99.38**	76.88	77.50	**100.00**	87.50	56.25
	C6	64.91	90.79	**93.42**	82.89	83.33	**96.05**	93.86	**98.25**	96.93
	C7	60.58	60.58	**92.31**	48.08	47.12	**67.31**	**99.04**	64.42	64.42
	C8	**42.56**	28.37	41.63	99.30	**100.00**	**100.00**	92.56	91.63	**97.91**
	C9	20.42	**22.93**	15.99	95.57	**100.00**	**100.00**	84.01	**100.00**	**100.00**
	C10	28.29	10.67	**50.87**	80.15	**98.76**	89.33	97.52	**98.76**	96.53
	C11	**97.85**	93.54	95.45	**100.00**	**100.00**	**100.00**	98.09	**100.00**	99.28
	C12	83.47	**90.04**	84.86	**96.61**	95.02	95.42	**100.00**	95.62	**100.00**
	C13	**100.00**	**100.00**	**100.00**	84.56	99.14	**100.00**	**100.00**	**100.00**	**100.00**
OA(%)		**70.62**	67.76	70.28	86.63	90.32	**90.38**	93.73	93.34	**94.31**
AA(%)		**65.38**	60.28	64.76	81.21	82.99	**83.61**	**92.62**	88.81	89.20
κ		**0.6725**	0.6400	0.6691	0.8513	0.8922	**0.8928**	0.9302	0.9259	**0.9366**

CapsGLOM has an accuracy of 18.73%. CapsGLOM also has its moments of poor performance, such that the accuracy is just 26.54% when classifying class 2 (Willow). For CapsViT, its recognition accuracy for class 4 is the lowest, which also causes it to lag behind the other two models in OA, AA, and κ. Although these three models may all overfit the training set, A2S2K-RN outperforms the other two classifiers on all three metrics. Therefore, we appreciate other excellent classifiers and do not emphasize that the two base classifiers in our proposed CAL framework are the best in all cases; in fact, they can be flexibly replaced by other excellent classifiers, as long as those classifiers can handle samples with the same form (dimension). One thing to note here is that none of the three models that we are comparing are lightweight models, so, when replacing any of the base classifiers in the CAL framework with a lightweight model, we recommend setting early stopping based on the training accuracies or training losses to avoid overfitting.

After two AL iterations, the improvement of CapsViT and CapsGLOM was better than that of A2S2K-RN. Moreover, CapsGLOM continued to maintain a significant advantage in classifying class 4 and achieved maximum values in three metrics. After four AL iterations, the performance of all the compared models has been significantly improved. The classification accuracy of CapsGLOM for the samples of class 4 is still leading. In addition, it is superior to the other two models in OA and κ, which also verifies the superiority of our proposed CapsGLOM model. CapsViT and A2S2K-RN have little difference in OA and κ, while A2S2K-RN has the highest AA among all the compared models,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

TABLE V
AL EXPERIMENTS ON THE KSC DATASET (THE SPATIAL SIZE OF TRAINING SAMPLES IS 7 × 7)

Class		Active Learning Iteration=0			Active Learning Iteration=2			Active Learning Iteration=4		
		A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM
Accuracies(%)	C1	93.29	**98.82**	95.39	96.97	96.97	**98.29**	97.76	98.16	**98.82**
	C2	**80.99**	62.81	32.23	93.80	**98.76**	**98.76**	**98.76**	97.52	98.35
	C3	87.45	80.00	**89.41**	**99.61**	94.51	99.22	91.37	94.12	**96.86**
	C4	**14.74**	7.97	13.15	**27.89**	12.75	17.13	29.08	29.88	**41.04**
	C5	51.88	55.00	**60.63**	**90.00**	88.75	80.62	**93.13**	81.88	86.88
	C6	67.11	55.70	**93.86**	**98.25**	77.63	90.79	**94.30**	62.72	79.39
	C7	**89.42**	60.58	74.04	35.58	25.00	**48.08**	**84.62**	42.31	63.46
	C8	1.86	14.19	**41.63**	**91.63**	66.98	82.56	93.72	96.05	**99.53**
	C9	15.41	**45.66**	18.30	44.51	**100.00**	98.65	91.71	75.34	**100.00**
	C10	35.48	12.66	**85.86**	92.56	76.43	**99.01**	97.77	**100.00**	**100.00**
	C11	91.63	91.87	**97.85**	**100.00**	98.80	**100.00**	99.52	**100.00**	**100.00**
	C12	65.34	**90.24**	87.65	**96.61**	87.25	94.02	94.82	**99.40**	**99.40**
	C13	88.77	99.57	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**
OA(%)		62.68	67.60	**74.01**	86.96	86.30	**91.38**	92.94	89.75	**94.61**
AA(%)		60.26	59.63	**68.46**	82.11	78.76	**85.16**	**89.74**	82.87	89.52
κ		0.5878	0.6379	**0.7115**	0.8550	0.8473	**0.9040**	0.9213	0.8856	**0.9400**

which also indicates that it has a more balanced classification performance for various categories. Finally, we show the classification maps obtained by each model after four AL iterations in Fig. 8(d)–(f).

Different spatial sizes of training samples will lead to different classification results. Meanwhile, different spatial sizes also mean different amounts of computation. To compare the performance of the proposed CAL framework under different sample sizes, we reconducted the experiment on the KSC dataset using 7 × 7 training samples. It should be noted that, when using 7 × 7 training samples, the PrimaryCaps layer in CapsViT will get 400 (16 × 5 × 5) PrimaryCaps instead of 784 (please refer to Table I). In Table V, we present the comparison results similar to Table IV. It can be seen that, when the initial 13 7 × 7 training samples are used, the overall performance of CapsViT does not change much; CapsGLOM's performance is improved to a certain extent, while the performance of A2S2K-RN drops off a lot. For class 8 (Graminoid), A2S2K-RN's classification accuracy (recall rate) is only 1.86%, which is hardly acceptable. After four AL iterations, CapsGLOM still keeps the lead in OA and κ, and A2S2K-RN still has the highest AA value. For this experiment, although A2S2K-RN is still superior to CapsViT in the end, CapsViT not only outperforms A2S2K-RN on the initial training set but also shows certain robustness to different spatial sizes of training samples.

For the effectiveness and superiority of CapsViT, we conducted an additional experiment based on the above 13 7 × 7 training samples. When we removed the ViT module from CapsViT, it obtained OA, AA, and κ values of 58.23%, 45.34%, and 0.5309, respectively. This precipitous performance attenuation is sufficient to demonstrate the success and importance of introducing ViT into CapsNet. The reason for this apparent difference is that, after the introduction of ViT, for each PrimaryCap, CapsViT can learn its global optimal representation, which makes a lot of sense for classification. To verify our claim, we pass a callback function into CapsViT to observe the output features of the PrimaryCaps layer and the ViT module. For each training sample, the output of both the PrimaryCaps layer and the ViT module is 400 16-D capsule vectors. In order to visualize the capsule features of each training sample, we first need to reduce the dimension of these features. Here, for each training sample, we use the PCA to project 400 16-D capsule vectors into a 3-D space in the direction preserving the maximum difference. Although the projected capsule vectors no longer have the property of being squashed, we can still use their molds to represent the probability of the occurrence of entity features.

The outputs of the PrimaryCaps layer and the ViT module for the 13 7 × 7 training samples are shown in Figs. 9 and 10, respectively. By comparing Figs. 9 and 10, we can find that the output of the PrimaryCaps layer is relatively discrete, which only focuses on the location and probability of the occurrence of various features, while the output of the ViT module is more uniform, and some features will become more obvious after considering the global correlation between all features.

Next, we examine the impact of different query strategies on our proposed CAL framework, and two query strategies based on EP and loss prediction are used for comparison. When comparing the query strategy based on loss prediction, we use the output of multiple layers in CapsViT to construct the loss prediction module. The layers used include the first convolution layer, the GN layer, the PrimaryCaps layer, and the ViT module. Among them, the output of the PrimayCaps layer and the ViT module needs to be reshaped into feature cubes first and then sent to the loss prediction module. It should be noted that, in our experiment, the gradient of the loss prediction module did not backpropagate to CapsViT. In addition, when calculating the losses of the loss prediction module, it requires that the number of samples in each minibatch must be an even number, which is usually not guaranteed for the last minibatch in each epoch. Thus, we made a small modification; when the number of samples in the minibatch is an odd number, we pair the first sample and the middle sample to calculate a loss value. When everything is ready, we give the experimental results based on different query strategies in Table VI, and the spatial size of the training samples is 9 × 9 (it will not be declared later).

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

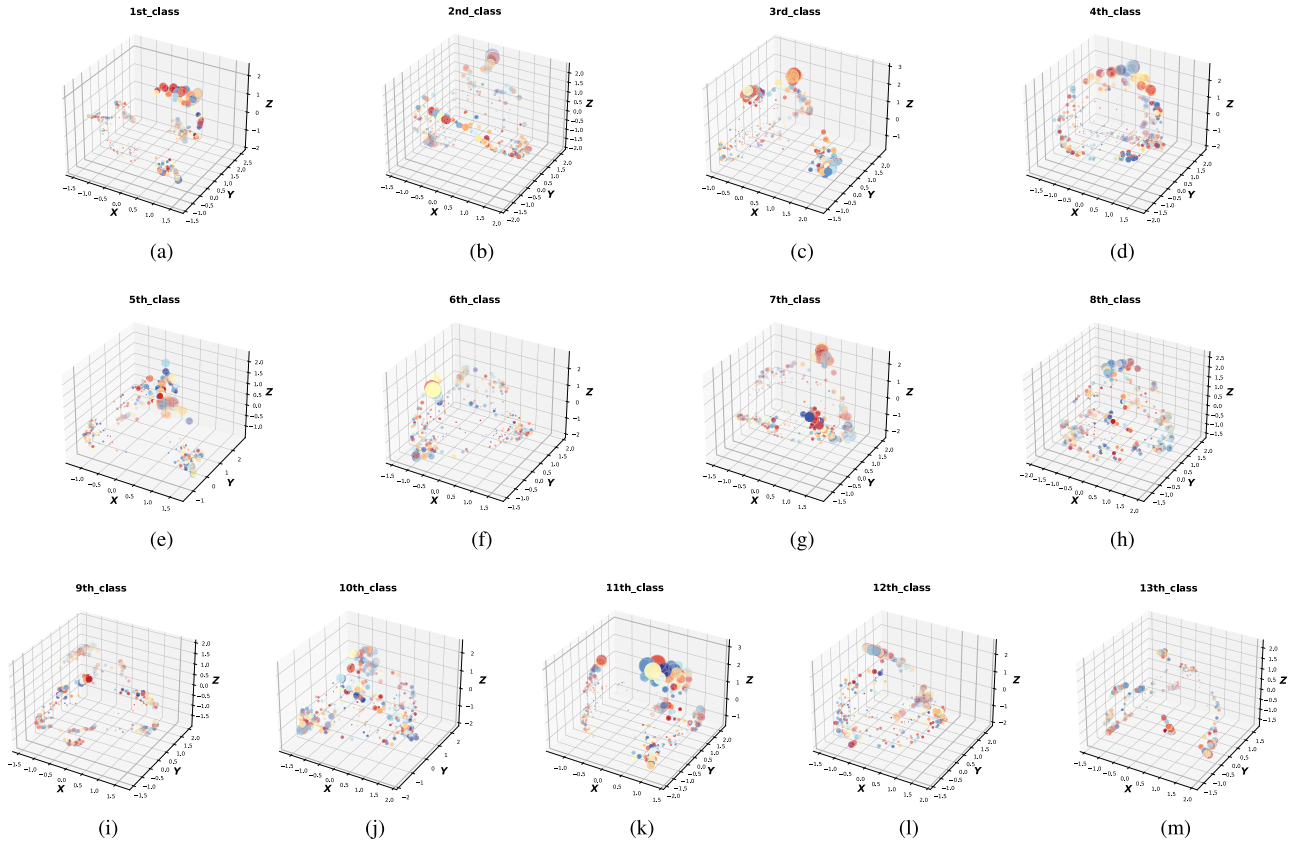


Fig. 9. Visualization of the PrimaryCaps layer output in CapsViT for the KSC dataset. Each dot represents a capsule feature, and the size of the dot indicates the probability of occurrence of the entity feature. (a)–(m) Features of 13 7 × 7 training samples.

TABLE VI
AL EXPERIMENTS ON THE KSC DATASET USING DIFFERENT QUERY STRATEGIES (THE SPATIAL SIZE OF TRAINING SAMPLES IS 9 × 9)

Class	BvSB-based			EP-Based			Loss Prediction-Based		
	A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM
OA(%)	93.73	93.34	**94.31**	91.57	94.52	**95.09**	89.44	89.28	**91.11**
AA(%)	**92.62**	88.81	89.20	88.72	90.27	**90.41**	86.18	83.00	**87.01**
κ	0.9302	0.9259	**0.9366**	0.9061	0.9390	**0.9454**	0.8823	0.8805	**0.9010**

By looking at Table VI, we can find that, although the results obtained by the heuristic based on loss prediction are acceptable, its performance is not as good as that based on BvSB and EP. In addition, CapsViT and CapsGLOM achieve better classification results when using EP-based query strategy, while A2S2K-RN does not. Meanwhile, when using EP-based heuristic, CapsGLOM obtained the best AA of 90.41%, but this value was still much lower than the AA of 92.62% achieved by A2S2K-RN using the BvSB-based query strategy. Therefore, on the whole, although the CAL framework obtains better classification results on the KSC dataset when adopting the EP-based query strategy, the BvSB-based query strategy is still the default choice of the CAL framework considering the substitutability of the two base classifiers in it.

Finally, to further verify the superiority of CapsViT and CapsGLOM, 10% of the labeled samples of each category were randomly selected and used as training samples to conduct supervised classification experiments. Deep models to be compared include HybridSN [4], CAEVT [6], A2S2K-RN [10], SpectralNET [13], SpectralFormer [20], SSFTT [23], and DMCapsNet [24]. For this series of experiments, we did not set up the random seed to control the selection of training samples and the initialization of each model to be compared, so the experiment was repeated ten times for each model to report AA and standard deviation. Besides, there are a few things to say.

1) When comparing CAEVT, we did not use the 3-D convolutional autoencoder to preprocess the original dataset for dimensionality reduction but used the ConvGRU module designed by us to learn the spectral features of all bands.
2) Except for SSFTT (it was trained 100 epochs), DMCapsNet (it was trained 100 epochs on the KSC and HU2013 datasets and 50 epochs on the PU and Chikusei datasets) and SpectralNET (it was trained 250 epochs on the KSC, PU, and Chikusei datasets and 200 epochs on the HU2013 dataset), all the compared models were trained 200 epochs, and the batch size was set to 32.
3) We did not divide the validation set separately to obtain the optimal parameters for each model because all models would not be severely overfit after 200 training epochs if enough training samples were available.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

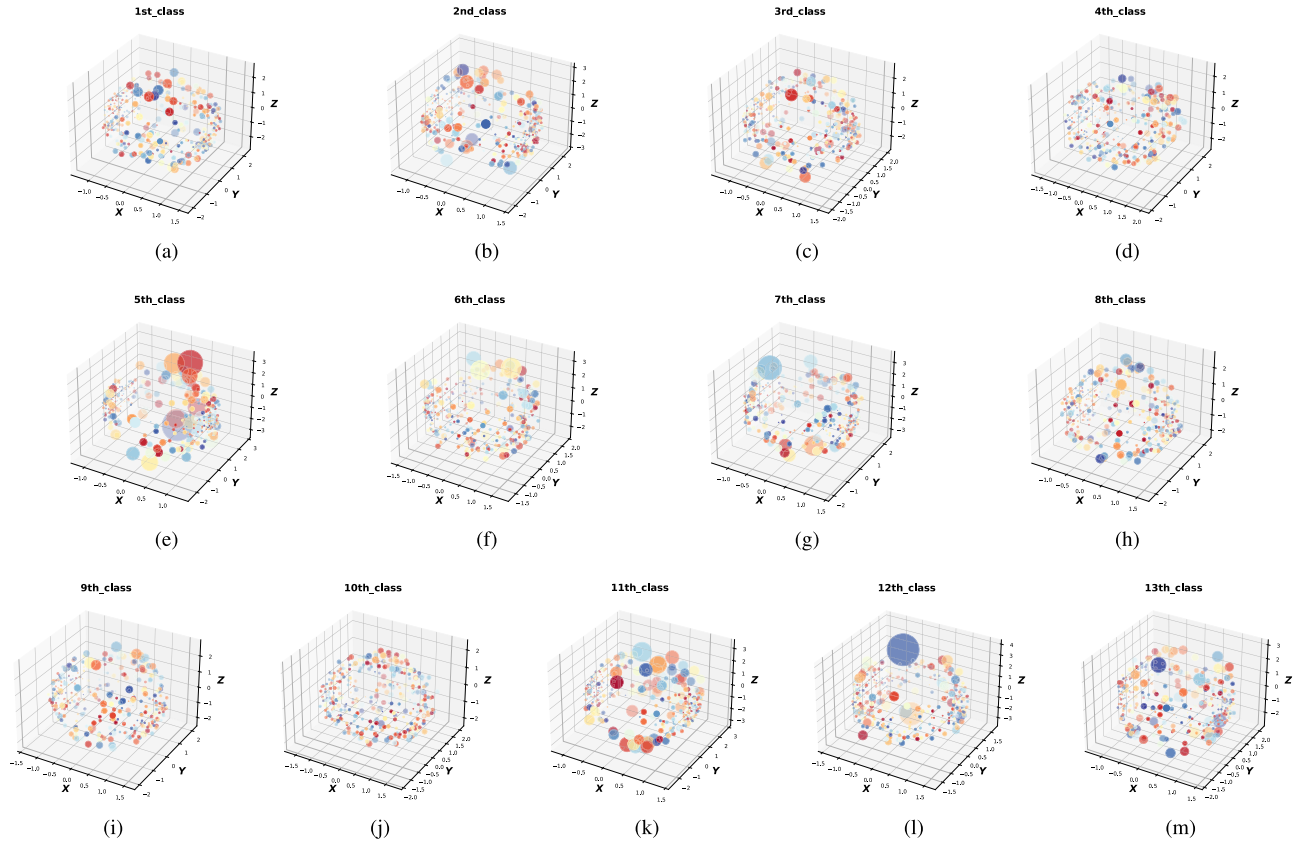


Fig. 10. Visualization of the ViT module output in CapsViT for the KSC dataset. Each dot represents a capsule feature, and the size of the dot indicates the probability of occurrence of the entity feature. (a)–(m) Features of 13 7 × 7 training samples.

4) To be fair, except HybridSN, DMCapsNet, and SpectralNET (who used 25 × 25, 25 × 25, and 24 × 24 training samples, respectively), the spatial size of training samples used by all models was 9 × 9.

The classification results obtained by different models are given in Table VII. It is clear that A2S2K-RN performed best on the KSC dataset when 10% of the labeled samples were used as training samples. HybridSN got the worst performance, probably due to its simplistic structure. Although both HybridSN and SpectralNET used training samples with a larger spatial size (such samples will contain more spatial information), the performance of SpectralNET is significantly better than that of HybridSN. In particular, for class 4, none of the other models had a recall rate of more than 90%, while SpectralNET had a recall rate of 90.71% with a relatively small deviation. In addition, SpectralFormer's performance was not satisfactory; compared with the highest OA value of 99.09% obtained by A2S2K-RN, the OA value obtained by SpectralFormer is only 94.64%, so the method that simply provides each unfolded band of the training sample patch to the ViT as an element of the input sequence is questionable. For CAEVT, although the MobileViT block it used is much less computative than ViT, its performance is not very good. As for SSFTT, it is a lightweight model, and its performance is medium. Moreover, although DMCapsNet uses viewpoint invariant transformation matrices to extract viewpoint invariant features and aggregates low-level capsule features by calculating the similarity between votes and distributions, its performance is mediocre compared to CapsViT. Our proposed CapsViT was 2.65%, 3.83%, and 0.0295 higher in OA, AA, and κ than DMCapsNet, respectively, which verifies the effectiveness and rationality of introducing ViT into CaspNet. Finally, by comparing all classification results, it can be found that CapsViT and CapsGLOM obtain the second and fourth best results, respectively, which is enough to show their superiority.

B. Experiment 2: Houston University 2013 Dataset

The HU2013 dataset was acquired by the National Center for Airborne Laser Mapping (NCALM) over the University of Houston campus and the neighboring urban area, and was released in the 2013 IEEE Geoscience and Remote Sensing Society (GRSS) Data Fusion Contest. The imagery has 144 spectral bands in the 380-nm region to the 1050-nm region, including 349 × 1905 pixels, and the spatial resolution is 2.5 m. This dataset contains 15 complex land-cover categories; the false-color image and ground truth are shown in Fig. 11(a) and (c).

For this dataset, there are 15 029 samples; we randomly selected five samples from each category to form the initial training set and set the number of AL iterations as 5, which means that a total of 75 samples were actively labeled by the expert. Besides, although the total number of samples in this dataset is relatively large, in order not to miss the most informative samples, we still did not divide the candidate set separately on this dataset. Compared with the experiments on

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
SUPERVISED CLASSIFICATION EXPERIMENTS ON THE KSC DATASET USING 10% TRAINING SAMPLES

Class		HybridSN	CAEVT	A2S2K-RN	DMCapsNet	SSFTT	SpectralNET	SpectralFormer	CapsViT	CapsGLOM
Accuracies(%)	C1	95.25±2.05	99.44±0.49	99.78±0.32	97.72±1.20	99.87±0.12	99.71±0.31	99.65±0.42	**99.85±0.24**	99.65±0.31
	C2	82.48±4.90	94.63±5.35	**99.13±1.97**	92.06±5.55	92.80±8.02	96.65±3.08	88.58±9.16	98.81±1.54	98.21±1.75
	C3	91.70±3.82	91.87±9.15	98.78±2.18	**99.52±1.16**	95.26±5.61	96.61±2.54	88.91±5.18	99.26±1.40	98.52±1.47
	C4	75.40±6.14	74.34±10.40	88.63±9.80	85.27±4.53	82.30±7.26	**90.71±5.73**	63.19±13.98	88.50±5.87	88.50±4.52
	C5	95.97±2.93	81.46±5.33	96.46±5.76	76.18±6.56	87.22±4.82	**97.15±4.60**	69.10±12.46	94.03±4.96	92.57±5.99
	C6	84.56±4.96	90.97±5.36	**98.79±1.79**	88.88±5.79	86.84±5.51	**98.79±1.41**	91.50±3.58	97.52±4.22	96.12±4.99
	C7	87.23±7.96	97.55±6.66	99.89±0.32	99.36±1.91	98.51±2.39	**100.00±0.00**	98.09±3.49	99.04±1.93	99.47±0.86
	C8	94.01±2.30	99.59±0.79	99.95±0.10	99.20±1.68	99.92±0.17	98.79±1.62	95.74±3.03	**100.00±0.00**	**100.00±0.00**
	C9	92.26±4.26	99.94±0.14	99.94±0.10	99.44±1.53	99.91±0.10	99.87±0.38	98.44±1.81	**100.00±0.00**	**100.00±0.00**
	C10	92.23±4.67	99.81±0.35	99.78±0.66	91.87±4.81	98.60±1.44	99.94±0.17	98.04±2.44	**100.00±0.00**	99.97±0.08
	C11	97.53±2.80	99.76±0.51	**100.00±0.00**	99.28±0.65	99.66±0.67	**100.00±0.00**	99.71±0.34	**100.00±0.00**	99.60±0.65
	C12	94.65±2.87	98.98±1.18	**99.62±0.92**	97.81±0.95	97.46±1.42	99.60±0.85	95.75±5.35	99.36±1.28	99.40±1.04
	C13	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
OA(%)		93.25±1.59	96.84±1.23	**99.09±0.50**	96.31±0.60	97.16±0.42	98.89±0.22	94.64±2.20	98.96±0.42	98.74±0.38
AA(%)		91.02±1.70	94.49±2.05	**98.52±0.93**	94.35±0.81	95.26±0.80	98.29±0.43	91.28±3.30	98.18±0.70	97.85±0.63
κ		0.9248±0.018	0.9648±0.014	**0.9899±0.006**	0.9589±0.007	0.9684±0.005	0.9876±0.003	0.9403±0.025	0.9884±0.005	0.9859±0.004

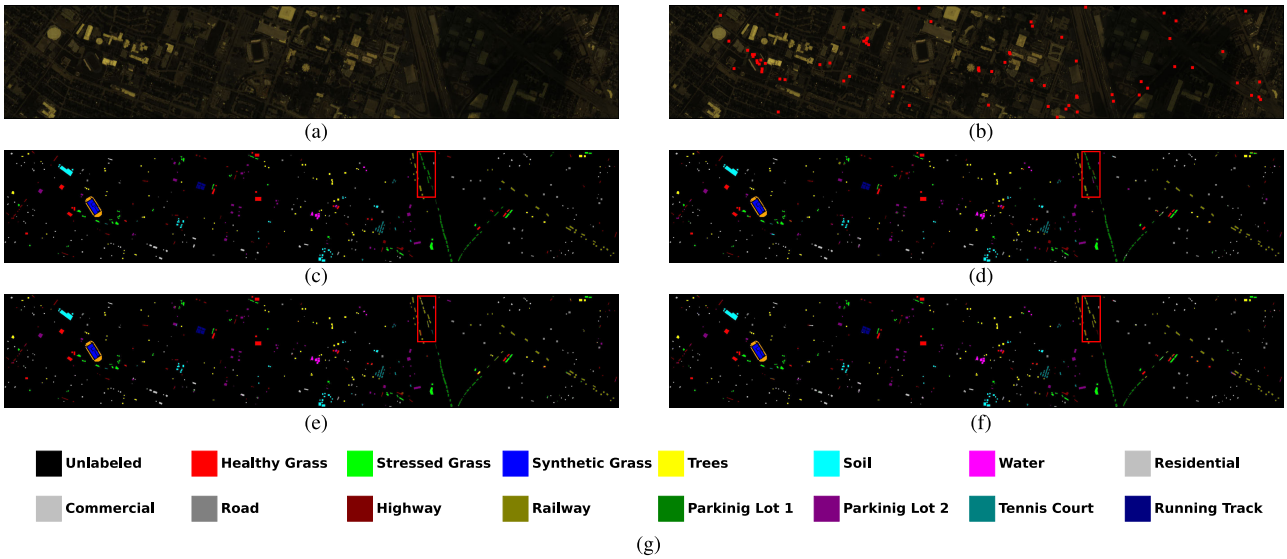


Fig. 11. Visualization of the classification maps obtained by different models on the HU2013 dataset. (a) Three-band false-color composite. (b) Training samples. (c) Ground truth. (d)–(f) Classification maps obtained by A2S2K-RN, CapsViT, and CapsGLOM after five AL iterations, respectively. (g) Legend.

the KSC dataset, we made two changes in the experiments on the HU2013 dataset: 1) the batch size was adjusted to 32 because there would not be a minibatch containing only one training sample and 2) considering that CapsViT and CapsGLOM may not be adequately trained on this dataset, we adjusted the training epochs for these two models to 250 instead of the default 200. Table VIII lists the classification results achieved by A2S2K-RN, CapsViT, and CapsGLOM on the initial training set and the training sets obtained after three and five AL iterations.

As can be seen from Table VIII, on the initial training set, CapsViT obtained the highest OA and κ, and A2S2K-RN obtained the highest AA. This result proves that CapsViT can obtain relatively ideal classification results when only a few training samples are available. Although CapsGLOM's performance was mediocre, it had the highest recall rates in five categories, which is helpful for its ability to assist experts in annotating. After three AL iterations, A2S2K-RN began to gain the upper hand, mainly because its recognition accuracy for class 11 (Railway) was better than that of CapsViT and CapsGLOM. After five AL iterations, we can find that A2S2K-RN achieved the best performance; its OA was 1.81% higher than CapsGLOM and 1.69% higher than CapsViT. At this point, it is obvious that there is a reason to replace CapsGLOM in our proposed CAL framework with A2S2K-RN. To compare the performance of the above three models from a visual perspective, the resulting classification maps are shown in Fig. 11(d)–(f). In Fig. 11, from the places highlighted by the red rectangles, we can see that A2S2K-RN is significantly better than CapsViT and CapsGLOM in classifying class 11. The experimental results in Table VIII are indeed the same; for class 11, the recall rates of the three models are, respectively, 86.34%, 66.67%, and 64.72%.

Next, we conducted supervised classification experiments on this dataset, and 10% of the samples in each category were selected as the training samples (a total of 1510 samples were selected). The experimental results obtained by different models are given in Table IX. As shown in Table IX, when the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
AL EXPERIMENTS ON THE HU2013 DATASET USING 75 INITIAL TRAINING SAMPLES

Class		Active Learning Iteration=0			Active Learning Iteration=3			Active Learning Iteration=5		
		A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM
Accuracies(%)	C1	94.54	93.58	**97.75**	**95.67**	85.96	90.69	98.48	94.54	**98.64**
	C2	79.02	**91.67**	70.62	90.56	**95.20**	98.32	96.08	99.68	**99.92**
	C3	**97.98**	96.68	95.09	**100.00**	99.57	**100.00**	**100.00**	**100.00**	**100.00**
	C4	94.67	**94.92**	92.01	99.11	99.19	**99.35**	97.82	**98.55**	92.33
	C5	91.67	**100.00**	99.84	**100.00**	99.92	99.27	99.19	**99.84**	98.71
	C6	**93.75**	88.75	85.00	**98.13**	91.56	88.44	**97.81**	94.69	95.63
	C7	**63.90**	39.67	46.95	**95.49**	87.17	89.31	**94.38**	90.26	92.72
	C8	46.81	45.76	**49.80**	66.67	**71.19**	68.60	78.93	84.75	**90.40**
	C9	72.89	76.18	**79.31**	**83.24**	79.15	74.66	**80.51**	78.99	73.62
	C10	49.10	**49.35**	42.23	76.68	67.51	**79.62**	**99.35**	88.54	90.75
	C11	50.08	48.13	**63.41**	**80.57**	56.67	66.26	**86.34**	66.67	64.72
	C12	56.03	**68.97**	52.20	84.45	**93.24**	88.19	86.16	**96.17**	95.28
	C13	**94.61**	92.03	94.40	**95.04**	94.83	92.24	**93.75**	92.67	92.89
	C14	**100.00**	94.09	99.53	**100.00**	99.76	**100.00**	**100.00**	**100.00**	99.53
	C15	97.56	**100.00**	**100.00**	**100.00**	**100.00**	99.69	**100.00**	**100.00**	98.63
OA(%)		74.57	**75.01**	73.93	**89.26**	86.00	87.44	**92.90**	91.21	91.09
AA(%)		**78.84**	78.65	77.88	**91.04**	88.06	88.98	**93.92**	92.36	92.25
κ		0.7255	**0.7304**	0.7184	**0.8839**	0.8487	0.8643	**0.9232**	0.9050	0.9037

TABLE IX
SUPERVISED CLASSIFICATION EXPERIMENTS ON THE HU2013 DATASET USING 10% TRAINING SAMPLES

Class		HybridSN	CAEVT	A2S2K-RN	DMCapsNet	SSFTT	SpectralNET	SpectralFormer	CapsViT	CapsGLOM
Accuracies(%)	C1	99.32±0.65	97.32±1.50	**99.65±0.54**	96.91±2.63	99.37±1.08	97.36±1.60	97.09±1.78	99.09±0.82	98.78±1.16
	C2	**99.80±0.16**	99.50±0.42	99.23±1.14	99.17±0.79	99.49±0.57	97.93±1.80	96.39±4.84	99.57±0.66	99.12±0.79
	C3	99.59±0.53	99.89±0.19	99.87±0.10	99.52±0.30	**99.94±0.11**	99.89±0.16	99.57±0.74	99.71±0.39	99.78±0.13
	C4	99.32±0.81	97.43±1.18	**99.74±0.36**	96.41±3.06	99.63±0.23	97.81±1.09	96.93±1.30	96.09±2.51	97.55±2.31
	C5	**100.00±0.00**	99.70±0.26	99.99±0.03	99.73±0.42	**100.00±0.00**	99.83±0.22	99.06±1.01	99.98±0.04	99.96±0.04
	C6	97.16±3.02	91.37±3.08	98.25±2.06	95.55±3.55	**98.49±2.47**	90.89±3.84	81.78±4.72	90.92±4.49	95.07±4.08
	C7	97.47±1.13	95.60±1.64	98.59±1.16	97.44±1.65	98.24±1.28	95.09±1.89	91.87±1.80	**98.82±0.51**	97.51±1.94
	C8	**97.47±1.65**	93.43±2.07	95.69±2.22	96.87±1.75	97.32±1.66	93.85±2.37	90.73±2.56	94.98±2.10	96.44±1.28
	C9	96.04±1.42	91.78±2.08	97.16±1.68	94.96±2.51	97.30±1.20	**97.33±0.87**	91.15±2.79	96.25±1.41	95.88±1.52
	C10	99.93±0.14	97.46±1.11	99.95±0.08	**100.00±0.00**	99.68±0.41	99.54±0.57	97.37±1.36	99.41±0.56	98.99±0.95
	C11	99.53±1.07	96.32±0.51	99.06±0.97	**99.89±0.10**	99.14±1.03	98.68±1.09	94.59±1.28	97.83±1.35	98.53±0.99
	C12	98.68±1.23	95.89±1.83	**99.14±0.56**	98.79±1.66	97.13±0.81	96.79±1.94	96.65±1.51	98.40±0.64	97.49±0.98
	C13	97.16±2.04	94.38±2.69	**98.08±1.72**	94.74±1.73	97.46±2.14	95.43±1.28	66.28±5.74	96.19±1.49	97.25±1.61
	C14	**100.00±0.00**	99.32±1.03	**100.00±0.00**	**100.00±0.00**	99.71±0.25	99.71±0.49	96.68±2.21	**100.00±0.00**	99.90±0.31
	C15	99.98±0.05	99.95±0.08	**100.00±0.00**	99.09±0.36	99.95±0.11	99.85±0.30	98.59±1.92	99.98±0.05	99.98±0.05
OA(%)		98.80±0.25	96.66±0.28	98.92±0.25	98.03±0.41	**98.99±0.24**	97.50±0.33	94.38±0.55	98.05±0.21	98.16±0.35
AA(%)		98.76±0.33	96.62±0.38	98.96±0.27	97.94±0.50	**98.99±0.29**	97.33±0.42	92.98±0.62	97.81±0.34	98.15±0.49
κ		0.9870±0.003	0.9639±0.003	0.9883±0.003	0.9787±0.004	**0.9890±0.003**	0.9729±0.004	0.9392±0.006	0.9789±0.002	0.9801±0.004

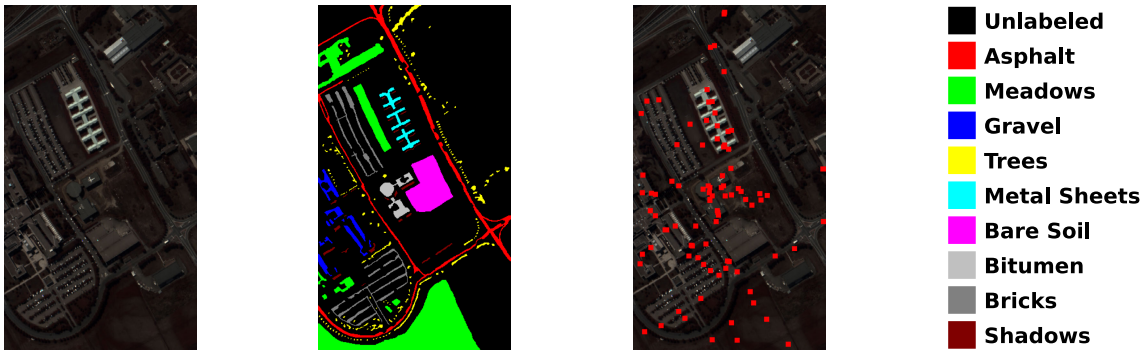
training samples are sufficient, SSFTT's performance on the HU2013 dataset is bright enough, followed by A2S2K-RN. Compared with the highest OA value of 98.99% obtained by SSFTT, our proposed CapsViT and CapsGLOM are also competitive, with OA values of 98.05% and 98.16%, respectively. Of course, the lightweight and high-performance SSFTT can guide us to improve CapsViT and CapsGLOM in future work. DMCapsNet achieved very close performance to CapsViT on the HU2013 dataset; compared with DMCapsNet, due to the introduction of the ViT module, CapsViT has stronger learning and expression ability; thus, the OA and κ values are 0.02% and 0.0002 higher than the former. However, DMCapsNet achieved 0.13% higher AA than CapsViT, which is worthy of affirmation. Meanwhile, the results obtained by CapsGLOM and CapsViT are close, and the difference between them is only 0.11% in terms of OA, which also proves that CapsGLOM can well learn the global correlation between various positions. Finally, although HybridSN's success may be due to the use of training samples with larger spatial sizes, its performance on the HU2013 dataset is worthy of praise. With an OA value of 94.38%, SpectralFormer is still pretty lackluster.

C. Experiment 3: Pavia University Dataset

The PU dataset was collected by the ROSIS (reflective optics system imaging spectrometer) over the PU in 2001. This dataset contains 103 spectral bands ranging from 430 to 860 nm, and the spatial resolution is 1.3 m. There are nine land-cover classes, and the images have a spatial dimension of 610 × 340. Fig. 12(a) and (b) shows the false-color image and the corresponding ground truth.

For this dataset, there are 42 776 samples; due to the high spatial resolution of this dataset, we randomly selected ten samples from each category to form the initial training set. The selected initial training samples are shown in Fig. 12(c).

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 12. PU dataset. (a) Three-band false-color composite. (b) Ground truth. (c) Training samples. (d) Legend.

TABLE X
AL EXPERIMENTS ON THE PU DATASET USING 90 INITIAL TRAINING SAMPLES

Class		Active Learning Iteration=0			Active Learning Iteration=3			Active Learning Iteration=5		
		A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM
Accuracies(%)	C1	87.21	87.24	**89.29**	**96.60**	91.69	92.07	92.98	88.69	**93.49**
	C2	**86.35**	84.25	83.93	**94.70**	88.72	94.54	**97.97**	94.67	97.45
	C3	**94.83**	72.52	86.55	**94.02**	79.22	90.33	**96.98**	92.10	80.33
	C4	**98.00**	93.91	88.47	98.13	**99.28**	96.66	**98.59**	96.95	97.02
	C5	99.63	**100.00**	99.48	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**
	C6	82.92	82.25	**92.19**	87.05	**93.33**	88.09	**93.52**	77.70	86.93
	C7	**93.86**	91.14	93.56	**98.03**	92.42	95.76	**99.24**	90.30	96.82
	C8	77.59	**86.96**	84.80	92.40	82.76	**94.83**	90.47	92.21	**95.51**
	C9	99.68	**99.79**	98.51	98.93	**99.57**	97.65	98.83	**99.89**	96.05
OA(%)		**87.52**	85.88	87.37	**94.47**	90.21	93.65	**96.15**	91.72	94.59
AA(%)		**91.12**	88.67	90.75	**95.54**	91.89	94.44	**96.51**	92.50	93.73
κ		**0.8376**	0.8166	0.8371	**0.9270**	0.8727	0.9161	**0.9490**	0.8903	0.9282

Since this dataset only contains 103 spectral bands, in the AL experiments, the training epochs of all compared models were uniformly adjusted to 150 to avoid overfitting. Considering the cost of expert annotation, the number of AL iterations was set to 5. Besides, in order to avoid training instability caused by too few samples in a minibatch, the batch size was set to 30 for all compared models. The key point is that, for the PU dataset, there are a large number of samples; to simulate the application in real scenarios, 10% (unlabeled) samples from each category were selected to construct the candidate sample set. The classification results obtained by A2S2K-RN, CapsViT, and CapsGLOM after different AL iterations are given in Table X.

As we can see, on the initial training set, A2S2K-RN performed the best, and CapsGLOM achieved similar results. Although CapsViT did not perform, as well as A2S2K-RN and CapsGLOM, it is acceptable. After three or five AL iterations, A2S2K-RN was still the best, followed by CapsGLOM, while CapsViT performed poorly. A2S2K-RN was able to perform so well mainly because it can well learn the abstract representation of spectral–spatial features. Of course, in this case, we can use A2S2K-RN to replace CapsViT. Finally, the classification maps obtained by A2S2K-RN, CapsViT, and CapsGLOM after five AL iterations are shown in Fig. 13.

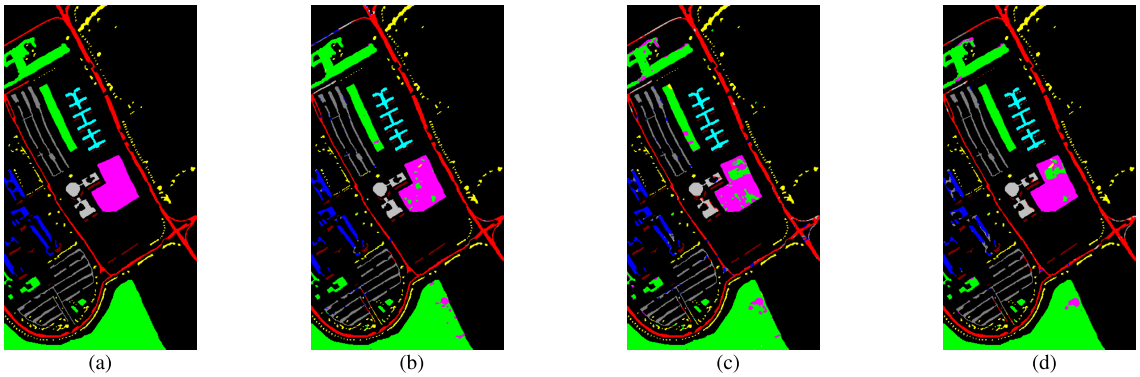
To verify the superiority of our proposed CapsViT and CapsGLOM, we conducted supervised classification experiments on the PU dataset. 2% of the samples in each category were selected as the training samples (a total of 858 samples were selected), and the experimental results are given in Table XI. It can be found that, when training samples were sufficient, although A2S2K-RN was still the one with the best performance, CapsViT and CapsGLOM proposed by us were the two most competitive models, and the difference between them and A2S2K-RN in OA is only 0.31%. One notable result is that CapsViT and CapsGLOM all achieved an average OA of 98.79%, but CapsViT achieved a higher AA value.

D. Experiment 4: Chikusei Dataset

The Chikusei dataset was taken by the Headwall Hyperspec-VNIR-C imaging sensor over agriculture and urban areas in Chikusei, Ibaraki, Japan, in 2014. This dataset has 128 bands in the spectral range from 363 to 1018 nm. The scene consists of 2517 × 2335 pixels with a spatial resolution of 2.5 m. Ground truth of 19 classes comprises water, three types of bare soil, seven types of vegetation, and eight types of man-made objects. Fig. 14(a) and (b) shows the false-color image and the corresponding ground truth.

For this dataset, there are 77 592 samples, and five samples from each category were randomly selected to construct the initial training set. In addition, 10% of the (unlabeled) samples from each category were randomly selected to construct the candidate sample set. In the AL experiments, to avoid overfitting, the training rounds of A2S2K-RN, CapsViT, and CapsGLOM were set to 150, 200, and 200, respectively. The batch size was set to 32. The classification results obtained by A2S2K-RN, CapsViT, and CapsGLOM after different

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 13. Classification maps obtained by different models after five AL iterations on the PU dataset. (a) Ground truth. (b)–(d) Classification maps obtained by A2S2K-RN, CapsViT, and CapsGLOM, respectively.

TABLE XI

SUPERVISED CLASSIFICATION EXPERIMENTS ON THE PU DATASET USING 2% TRAINING SAMPLES

Class		HybridSN	CAEVT	A2S2K-RN	DMCapsNet	SSFTT	SpectralNET	SpectralFormer	CapsViT	CapsGLOM
Accuracies(%)	C1	**99.12±0.58**	95.56±2.97	99.12±0.57	97.68±0.70	98.97±0.33	93.64±1.83	89.06±1.71	98.84±0.56	98.82±0.68
	C2	**99.91±0.13**	99.88±0.06	99.77±0.26	99.86±0.08	99.73±0.22	98.85±0.26	98.58±0.56	99.88±0.16	99.86±0.10
	C3	93.12±2.07	88.23±3.15	**94.90±2.37**	93.53±3.29	85.38±3.99	77.37±5.48	68.81±5.97	89.69±3.43	92.63±3.99
	C4	95.16±1.33	97.77±1.16	**98.75±0.95**	84.57±1.63	97.54±1.73	93.63±2.08	94.56±1.57	97.94±0.56	97.48±0.73
	C5	99.81±0.29	99.91±0.14	99.95±0.08	99.33±0.59	**99.99±0.02**	98.81±1.91	99.86±0.17	99.98±0.05	99.58±0.74
	C6	**99.97±0.04**	95.66±3.15	99.67±0.55	99.92±0.11	98.02±1.68	94.63±2.46	86.42±3.39	99.71±0.38	99.81±0.27
	C7	99.29±0.83	89.55±3.49	96.65±2.34	**99.31±0.82**	98.99±0.91	91.42±1.64	72.80±5.35	95.73±2.63	95.59±2.80
	C8	96.14±2.01	94.85±1.01	97.97±1.66	95.06±1.84	96.48±1.44	93.81±2.21	79.61±2.12	**98.14±1.44**	97.46±1.44
	C9	87.80±4.26	**99.89±0.16**	99.80±0.23	81.77±4.18	98.74±0.96	85.53±10.47	96.83±1.66	99.82±0.21	98.36±1.99
OA(%)		98.51±0.30	97.24±0.67	**99.10±0.29**	97.27±0.17	98.23±0.41	95.16±0.45	91.50±1.00	98.79±0.22	98.79±0.24
AA(%)		96.70±0.58	95.70±0.84	**98.51±0.41**	94.56±0.44	97.09±0.64	91.97±1.51	87.39±1.37	97.75±0.39	97.73±0.58
κ		0.9802±0.004	0.9633±0.009	**0.9881±0.004**	0.9638±0.002	0.9766±0.005	0.9358±0.006	0.8867±0.013	0.9839±0.003	0.9839±0.003

TABLE XII

AL EXPERIMENTS ON THE CHIKUSEI DATASET USING 95 INITIAL TRAINING SAMPLES

Metric	Active Learning Iteration=0			Active Learning Iteration=3			Active Learning Iteration=5		
	A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM	A2S2K-RN	CapsViT	CapsGLOM
OA(%)	93.03	**93.80**	93.01	96.16	95.89	**98.14**	**99.55**	98.73	98.77
AA(%)	93.19	92.95	**93.85**	96.30	96.08	**96.54**	**99.56**	97.08	98.50
κ	0.9196	**0.9286**	0.9194	0.9558	0.9526	**0.9786**	**0.9948**	0.9853	0.9858

AL iterations are given in Table XII.[6] By comparing the experimental results, we can find that CapsViT obtained the highest OA and Kappa, and CapsGLOM obtained the highest AA on the initial training set. This finding again shows that CapsViT and CapsGLOM have better classification performance when training samples are scarce. After three AL steps, CapsGLOM showed the largest performance improvement, and it also performed the best. After five AL steps, although the performance of CapsViT and CapsGLOM is inferior to that of A2S2K-RN, satisfactory classification results are obtained, which indicates that the CAL scheme designed by us is successful. It is worth noting that, although the proposed CAL framework can improve the OA of CapsViT and CapsGLOM from 93.80% and 93.01% to 98.73% and 98.77%, respectively, and requires only five AL iterations, the optimal AL iterations depend on the budget of expert annotation. Finally, we present the panoramic classification maps obtained by A2S2K-RN, CapsViT, and CapsGLOM in Fig. 14(d)–(f). The panoramic classification map can better depict the landscape of the scene, which depends on the generalization ability of the classifier. By looking at Fig. 14(d)–(f), we can see that, because A2S2K-RN has better classification performance, the panoramic classification map obtained by it has better texture details. From this, we can conclude that, although CapsViT and CapsGLOM are not the best classifiers, the samples actively selected by CAL can also be used to train other excellent classifiers, which exactly corresponds to the core idea of AL, data efficiency.

To fully measure the classification performance of CapsViT and CapsGLOM, we conducted supervised classification experiments on the Chikusei dataset; 30 samples from each category were randomly selected to build the training set, and the remaining samples were used as test samples. The classification results of all the compared models are given in Table XIII. By observing the results, we can see that, while A2S2K-RN remained in the lead and DMCapsNet followed with the second-best results, CapsViT and CapsGLOM were the best-performing models after A2S2K-RN and DMCapsNet. Besides that, although we use more training samples to

[6]For the sake of compactness, we have not given the per-class accuracies, and interested readers can reproduce our experiments for themselves.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

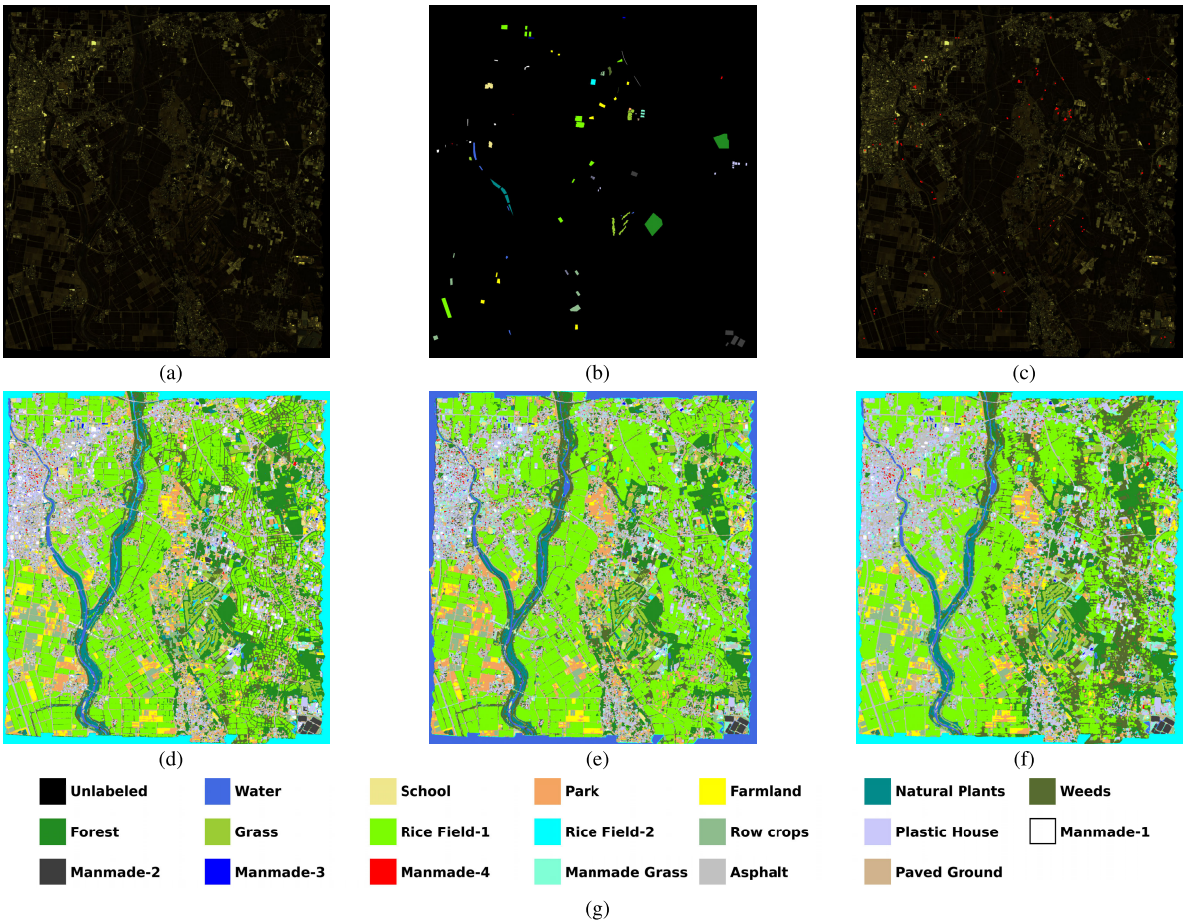


Fig. 14. Visualization of the classification maps obtained by different models on the Chikusei dataset. (a) Three-band false-color composite. (b) Ground truth. (c) Training samples. (d)–(f) Classification maps obtained by A2S2K-RN, CapsViT, and CapsGLOM after five AL iterations, respectively. (g) Legend.

TABLE XIII
SUPERVISED CLASSIFICATION EXPERIMENTS ON THE CHIKUSEI DATASET USING 30 TRAINING SAMPLES PER-CLASS

Metric	HybridSN	CAEVT	A2S2K-RN	DMCapsNet	SSFTT	SpectralNET	SpectralFormer	CapsViT	CapsGLOM
OA(%)	98.14±0.63	98.45±0.47	**99.12±0.40**	98.85±0.58	98.60±0.56	94.48±1.04	96.75±0.70	98.76±0.31	98.77±0.25
AA(%)	98.67±0.47	98.42±0.34	**99.22±0.38**	98.90±0.62	98.81±0.38	94.90±0.88	95.99±0.48	98.69±0.52	98.72±0.21
κ	0.9785±0.007	0.9821±0.005	**0.9898±0.005**	0.9868±0.007	0.9839±0.006	0.9365±0.012	0.9625±0.008	0.9856±0.004	0.9858±0.003
Training(s)	41.30	123.39	128.09	168.45	11.77	147.97	139.77	401.63	238.73
Testing(s)	7.23	18.47	20.00	97.82	4.28	31.24	22.24	94.18	60.13

train CapsGLOM, the average OA and κ values obtained by it, namely, 98.77% and 0.9858, are consistent with those obtained by the CAL framework after five AL iterations, demonstrating the effectiveness of our designed CAL framework.

Finally, to clearly understand the complexity of each model being compared, we also give their average training and testing times in Table XIII. From this, we can see that, since the model construction of SSFTT is the most lightweight, it consumes the least time. In addition, HybridSN also does not require much time due to its relatively simple structure. Because DMCapsNet needs to train the viewpoint invariant transformation matrices and the costs of whether the capsules are activated through the EM iterative algorithm, it also takes a lot of time to train and test. Owing to the introduction of the ViT module, CapsViT requires more time to train and predict. In addition, although CapsGLOM is very close to CapsViT in performance, the time required is moderate among all models. Incidentally, when CapsViT is applied to training samples with larger spatial sizes, it is better to use stride convolution first to reduce the spatial size; otherwise, too many PrimaryCaps will be constructed, resulting in increased model complexity.

V. DISCUSSION

To clearly understand the effectiveness, sensitivity, and complexity of the developed CapsViT and CapsGLOM, we conducted extensive analysis experiments. All the analysis experiments were performed on the PU dataset as it is one of the most classic and widely used datasets in the remote sensing community.

A. Ablation Study

To further confirm the effectiveness of each component in CapsViT and CapsGLOM, we conducted ablation studies.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

TABLE XIV
ABLATION EXPERIMENTS ON THE PU DATASET

Method	ConvGRU	ViT	Trainable Parameters	Cost($10^6\times$FLOPs)	Training Time(s)	OA	AA	κ
CapsViT w/o ConvGRU and ViT			1,103,168	12.658	70.64	98.53	97.41	0.9805
CapsViT w/o ConvGRU		✓	4,804,616	50.943	286.53	98.85	98.10	0.9848
CapsViT w/o ViT	✓		1,133,936	16.877	162.43	98.80	98.09	0.9841
CapsViT	✓	✓	4,835,384	55.162	340.40	98.89	98.07	0.9852
CapsGLOM w/o ConvGRU		–	336,507	26.713	81.60	98.82	98.03	0.9844
CapsGLOM	✓	–	413,355	32.895	192.44	98.85	98.03	0.9848

In this part, in order to eliminate the influence of different training samples on each ablation method, we used a random seed (seed = 0) to select fixed training samples. Other than that, to eliminate the effect of model initialization on experimental results, all ablation experiments were performed ten times to report more stable and reliable average results. As with the supervised classification experiments on the PU dataset, we randomly took 2% samples from the ground truth as the training samples, and all the ablation methods were trained for 200 epochs.

Table XIV presents the results of the ablation experiments, in which OA, AA, and κ all represent the average values of ten experiments. For CapsViT, when we removed both ConvGRU and ViT, its performance deteriorated significantly. When ViT is removed, the model corresponds to the conventional CapsNet plus the ConvGRU module. We can see that it achieves a performance not far from the original model and has a 0.27% improvement in OA compared to the method with both ConvGRU and ViT removed, which proves the effectiveness of the ConvGRU module. When we removed ConvGRU, its performance did not drop much, and the OA improved by 0.32% compared to the method with both ConvGRU and ViT removed, thus proving the effectiveness of the ViT module. In addition, we can observe that ViT brings more improvements to CapsViT than ConvGRU. For CapsGLOM, when ConvGRU was removed, the performance degraded but not significantly, which proves that the proposed CapsGLOM is successful and the ConvGRU module is effective.

To better observe the complexity of CapsViT and CapsGLOM and their components, we also give the trainable parameter number, floating-point operations (FLOPs), and average training time of each ablation method in Table XIV. The results indicate that ViT occupies the main computation of CapsViT. Therefore, we suggest that, when applying CapsViT to training samples with large space sizes, it is better to use step convolution to reduce the space size; otherwise, too many PrimaryCaps will be constructed, resulting in increased model complexity. In contrast, CapsGLOM is relatively lightweight, which also hints at its unlimited potential and charm.

B. Impact of the Number of Training Samples

To investigate the impact of different numbers of training samples on CapsViT and CapsGLOM, we performed extended experiments on the PU dataset. Specifically, we randomly selected ten to 100 training samples from each class to observe the performance changes of CapsViT and CapsGLOM. To control the variables in the experiments and simply compare CapsViT and CapsGLOM, we used the random seed (seed = 0) to select training samples and control the initialization of CapsViT and CapsGLOM. The results are plotted in Fig. 15.

It can be seen from the figure that, with the increase in training samples, the performance of CapsViT and CapsGLOM generally shows an upward trend. When the number of training samples in each category reached 60, the upward trend was relatively stable. This is because, although the number of training samples for each category has increased, the additional samples are not really informative and may contain a lot of duplicate information. As we have seen in the AL experiments on the PU dataset, although the initial samples of each category are only 10, after five AL steps, that is, 45 samples are actively labeled, the OA of CapsViT and CapsGLOM can reach 91.72% and 94.59%, respectively; at this time, the proportion of training samples was only 0.3256%. However, to exceed the OA of 91.72%, CapsViT may need to randomly select 18 samples from each category (0.38%); to exceed the OA of 94.59%, CapsGLOM probably needs to randomly select 20 samples from each category (0.42%).

Finally, it is important to note that, although the default training rounds of CapsViT and CapsGLOM are 200, in practical supervised classification applications, we recommend setting a larger training round and then determining whether to stop early to ensure optimal model acquisition based on the accuracy or loss on the validation set.

C. Impact of the Reconstruction Loss Weight in CapsViT

As can be seen in (9), the reconstruction loss weight α is used as a regularization coefficient to constrain CapsViT so that it has certain reconstruction ability. Too large weight will make the model pay more attention to its coding ability, while too small weight will weaken the model's reconstruction ability and lose the regularization constrain. To study the effect of different reconstruction loss weights on CapsViT performance, we conducted comparative experiments on the PU dataset. We chose the value of α from the set of {0.001, 0.005, 0.01, 0.05, 0.1}. In addition, to control the experimental variables, we used the random seed to control the selection of training samples (2%) and the initialization of the model. Fig. 16 shows the performance of CapsViT when using different values of α over the PU dataset.

By observing the bar graph, it can be found that different α values will lead to different performance, and when it was set to 0.01, CapsViT achieves the best performance. Although most of the performance differences are less than 0.2%,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

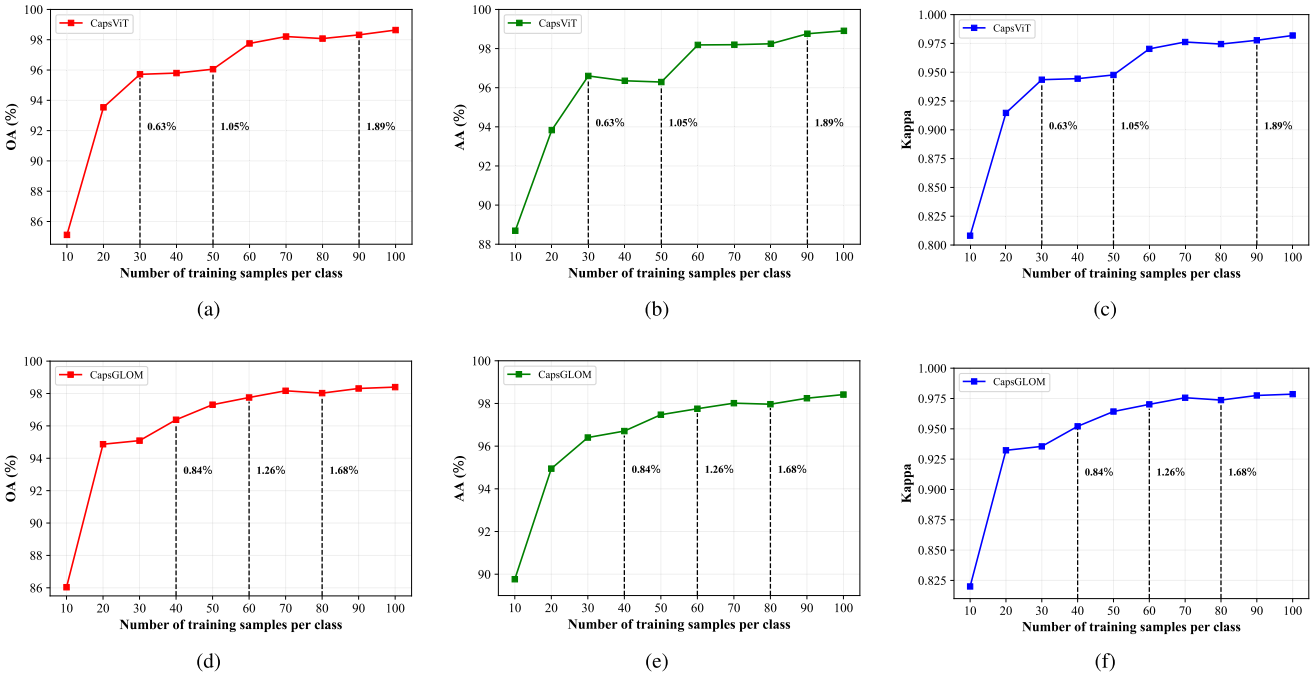


Fig. 15. Performance of CapsViT and CapsGLOM with different numbers of training samples on PU dataset. (a)–(c) Performance of CapsViT. (d)–(f) Performance of CapsGLOM. The percentage near the dashed line is the percentage of training samples.

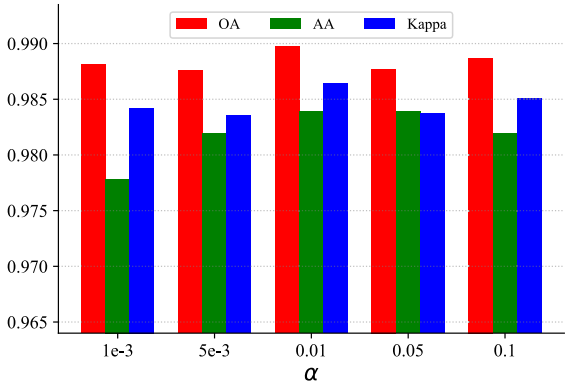


Fig. 16. Sensitivity of the reconstruction loss weight α in CapsViT on the PU dataset.

considering that, in many practical applications, the high-level representation of the instance is of great significance, the weight of the reconstruction loss as an important hyperparameter needs to be carefully adjusted. Of course, 0.01 is just the default value for α, and when CapsViT is deployed in a real application, we recommend using a grid search to assign an appropriate value to α.

VI. Conclusion

In this article, we propose a CAL framework based on improved CapsNets for HIC. The proposed CAL framework includes two excellent base classifiers and a CAL scheme that considers both the diversity of actively selected samples and the cost of expert annotation. The first base classifier is CapsNet combined with ViT, which is named CapsViT. By introducing ViT, CapsViT can better learn the global correlation between various PrimaryCaps and improve classification performance. The second base classifier is designed based on the improved GLOM system and is named CapsGLOM. Instead of using the preassigned universal capsule to learn the multilevel embeddings at various positions, CapsGLOM borrowed from the way CapsNet builds PrimaryCaps and lets the network learn these embeddings itself. AL experiments on four benchmark HSI datasets show that our proposed CapsViT and CapsGLOM can obtain good or even the best classification performance on the initial training set, which puts the way for the subsequent active selection of the most informative samples. At the same time, the proposed CAL framework is flexible, and any excellent deep model can be used to replace either of the two base classifiers that we designed, as long as it can handle samples with the same form (dimension). In addition, compared with advanced deep models, our proposed CapsViT and CapsGLOM can obtain very competitive results in supervised classification experiments on four datasets.

Finally, our future work is mainly reflected in the following three aspects.

1) CapsViT introduces ViT on the basis of the original CapsNet, resulting in relatively high complexity. How to reduce the complexity of CapsViT will be meaningful work.
2) Given the excellent performance of lightweight SSFTT on the HU2013 dataset and DMCapsNet on the Chikusei dataset, their advantages can be used to improve CapsViT and CapsGLOM.
3) Improving the performance of CapsGLOM from the perspective of interpretability will aid future research in HIC.

References

[1] D. Datta, P. K. Mallick, A. K. Bhoi, M. F. Ijaz, J. Shafi, and J. Choi, "Hyperspectral image classification: Potentials, challenges, and future directions," *Comput. Intell. Neurosci.*, vol. 2022, pp. 1–36, Apr. 2022.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

[2] X. Cao, J. Yao, Z. Xu, and D. Meng, "Hyperspectral image classification with convolutional neural network and active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 7, pp. 4604–4616, Jul. 2020.

[3] Y. Chen, L. Huang, L. Zhu, N. Yokoya, and X. Jia, "Fine-grained classification of hyperspectral imagery based on deep learning," *Remote Sens.*, vol. 11, no. 22, p. 2690, Nov. 2019.

[4] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[5] Y. Cai, X. Liu, and Z. Cai, "BS-nets: An end-to-end framework for band selection of hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 3, pp. 1969–1984, Mar. 2020.

[6] Z. Zhang, T. Li, X. Tang, X. Hu, and Y. Peng, "CAEVT: Convolutional autoencoder meets lightweight vision transformer for hyperspectral image classification," *Sensors*, vol. 22, no. 10, p. 3902, May 2022.

[7] Z. Zheng, Y. Zhong, A. Ma, and L. Zhang, "FPGA: Fast patch-free global learning framework for fully end-to-end hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5612–5626, Aug. 2020.

[8] Q. Zhu et al., "A spectral–spatial-dependent global learning framework for insufficient and imbalanced hyperspectral image classification," *IEEE Trans. Cybern.*, vol. 52, no. 11, pp. 11709–11723, Nov. 2022.

[9] L. Wang, H. Wang, L. Wang, X. Wang, Y. Shi, and Y. Cui, "RSSGL: Statistical loss regularized 3-D ConvLSTM for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5529420.

[10] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-based adaptive spectral–spatial kernel ResNet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7831–7843, Sep. 2021.

[11] M. E. Paoletti et al., "Capsule networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2145–2160, Apr. 2019.

[12] X. Tang et al., "Hyperspectral image classification based on 3-D octave convolution with spatial–spectral attention network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2430–2447, Mar. 2021.

[13] T. Chakraborty and U. Trehan, "SpectralNET: Exploring spatial–spectral WaveletCNN for hyperspectral image classification," 2021, *arXiv:2104.00341*.

[14] W.-S. Hu, H.-C. Li, L. Pan, W. Li, R. Tao, and Q. Du, "Spatial–spectral feature extraction via deep ConvLSTM neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 6, pp. 4237–4250, Jun. 2020.

[15] W.-S. Hu, H.-C. Li, Y.-J. Deng, X. Sun, Q. Du, and A. Plaza, "Lightweight tensor attention-driven ConvLSTM neural network for hyperspectral image classification," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 3, pp. 734–745, Apr. 2021.

[16] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[17] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30. Red Hook, NY, USA: Curran Associates, 2017, pp. 6000–6010.

[18] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[19] X. He, Y. Chen, and Z. Lin, "Spatial–spectral transformer for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 3, p. 498, Jan. 2021.

[20] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5518615.

[21] Z. Zhao, D. Hu, H. Wang, and X. Yu, "Convolutional transformer network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[22] D. Wang, J. Zhang, B. Du, L. Zhang, and D. Tao, "DCN-T: Dual context network with transformer for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 32, pp. 2536–2551, 2023.

[23] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5522214.

[24] A. Ravikumar, P. N. Rohit, M. K. Nair, and V. Bhatia, "Hyperspectral image classification using deep matrix capsules," in *Proc. Int. Conf. Data Sci., Agents Artif. Intell. (ICDSAAI)*, vol. 1, Dec. 2022, pp. 1–7.

[25] G. E. Hinton, S. Sabour, and N. Frosst, "Matrix capsules with EM routing," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–15.

[26] S. S. Sawant and M. Prabukumar, "Semi-supervised techniques based hyper-spectral image classification: A survey," in *Proc. Innov. Power Adv. Comput. Technol. (i-PACT)*, Apr. 2017, pp. 1–8.

[27] C. Ding et al., "Boosting few-shot hyperspectral image classification using pseudo-label learning," *Remote Sens.*, vol. 13, no. 17, p. 3539, Sep. 2021.

[28] R. Hang, F. Zhou, Q. Liu, and P. Ghamisi, "Classification of hyperspectral images via multitask generative adversarial networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 2, pp. 1424–1436, Feb. 2021.

[29] H. Liang, W. Bao, X. Shen, and X. Zhang, "Spectral–spatial attention feature extraction for hyperspectral image classification based on generative adversarial network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 10017–10032, 2021.

[30] H. Wu and S. Prasad, "Semi-supervised deep learning using pseudo labels for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 27, no. 3, pp. 1259–1270, Mar. 2018.

[31] B. Fang, Y. Li, H. Zhang, and J. C.-W. Chan, "Collaborative learning of lightweight convolutional neural network and deep clustering for hyperspectral image semi-supervised classification with limited training samples," *ISPRS J. Photogramm. Remote Sens.*, vol. 161, pp. 164–178, Mar. 2020.

[32] M. Seydgar, S. Rahnamayan, P. Ghamisi, and A. A. Bidgoli, "Semisupervised hyperspectral image classification using a probabilistic pseudo-label generation framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5535218.

[33] H. Zeng, Q. Liu, M. Zhang, X. Han, and Y. Wang, "Semi-supervised hyperspectral image classification with graph clustering convolutional networks," 2020, *arXiv:2012.10932*.

[34] S. Jia et al., "A semisupervised Siamese network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5516417.

[35] I. J. Goodfellow et al., "Generative adversarial nets," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, vol. 2, 2014, pp. 2672–2680.

[36] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," 2017, *arXiv:1701.07875v3*.

[37] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of Wasserstein GANs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–9.

[38] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath, "Generative adversarial networks: An overview," *IEEE Signal Process. Mag.*, vol. 35, no. 1, pp. 53–65, Jan. 2018.

[39] Z. Wang, Q. She, and T. E. Ward, "Generative adversarial networks in computer vision: A survey and taxonomy," *ACM Comput. Surveys*, vol. 54, no. 2, pp. 1–38, Mar. 2022.

[40] X. Chen, Y. Duan, R. Houthooft, J. Schulman, I. Sutskever, and P. Abbeel, "InfoGAN: Interpretable representation learning by information maximizing generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1–9.

[41] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," 2018, *arXiv:1802.05957*.

[42] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5046–5063, Sep. 2018.

[43] Y. Zhan, D. Hu, Y. Wang, and X. Yu, "Semisupervised hyperspectral image classification based on generative adversarial networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 2, pp. 212–216, Feb. 2018.

[44] J. Feng, X. Feng, J. Chen, X. Cao, X. Zhang, L. Jiao, and T. Yu, "Generative adversarial networks based on collaborative learning and attention mechanism for hyperspectral image classification," *Remote Sens.*, vol. 12, no. 7, pp. 212–216, 2020.

[45] C. Tao, H. Wang, J. Qi, and H. Li, "Semisupervised variational generative adversarial networks for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 914–927, 2020.

[46] X. Wang, K. Tan, Q. Du, Y. Chen, and P. Du, "CVA2E: A conditional variational autoencoder with an adversarial training process for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5676–5692, Aug. 2020.

[47] Z. Chen, L. Tong, B. Qian, J. Yu, and C. Xiao, "Self-attention-based conditional variational auto-encoder generative adversarial networks for hyperspectral classification," *Remote Sens.*, vol. 13, no. 16, p. 3316, Aug. 2021.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.

[48] Z. Li, X. Zhu, Z. Xin, F. Guo, X. Cui, and L. Wang, "Variational generative adversarial network with crossed spatial and spectral interactions for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 16, p. 3131, 2021.

[49] W.-Y. Wang, H.-C. Li, Y.-J. Deng, L.-Y. Shao, X.-Q. Lu, and Q. Du, "Generative adversarial capsule network with ConvLSTM for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 3, pp. 523–527, Mar. 2021.

[50] X. Wang, K. Tan, Q. Du, Y. Chen, and P. Du, "Caps-TripleGAN: GAN-assisted CapsNet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 7232–7245, Sep. 2019.

[51] J. Wang, S. Guo, R. Huang, L. Li, X. Zhang, and L. Jiao, "Dual-channel capsule generation adversarial network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501016.

[52] W. Liu, J. You, and J. Lee, "HSIGAN: A conditional hyperspectral image synthesis method with auxiliary classifier," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 3330–3344, 2021.

[53] Y. Cao, Y. Wang, J. Peng, C. Qiu, L. Ding, and X. X. Zhu, "SDFL-FC: Semisupervised deep feature learning with feature consistency for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 12, pp. 10488–10502, Dec. 2021.

[54] W. Zhao, X. Chen, J. Chen, and Y. Qu, "Sample generation with self-attention generative adversarial adaptation network (SaGAAN) for hyperspectral image classification," *Remote Sens.*, vol. 12, no. 5, p. 843, Mar. 2020.

[55] J. Feng, H. Yu, L. Wang, X. Cao, X. Zhang, and L. Jiao, "Classification of hyperspectral images based on multiclass spatial–spectral generative adversarial networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5329–5343, Aug. 2019.

[56] S. Zhang, X. Zhang, T. Li, H. Meng, X. Cao, and L. Wang, "Adversarial representation learning for hyperspectral image classification with small-sized labeled set," *Remote Sens.*, vol. 14, no. 11, p. 2612, May 2022.

[57] J. Wang, F. Gao, J. Dong, and Q. Du, "Adaptive DropBlock-enhanced generative adversarial networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5040–5053, Jun. 2021.

[58] F. Zhang, J. Bai, J. Zhang, Z. Xiao, and C. Pei, "An optimized training method for GAN-based hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 10, pp. 1791–1795, Oct. 2021.

[59] J. Shi, T. Shao, X. Liu, X. Zhang, Z. Zhang, and Y. Lei, "Evolutionary multitask ensemble learning model for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 936–950, 2021.

[60] S. Hao, W. Wang, Y. Ye, E. Li, and L. Bruzzone, "A deep network architecture for super-resolution-aided hyperspectral image classification with classwise loss," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4650–4663, Aug. 2018.

[61] Y. Zhang, W. Li, M. Zhang, S. Wang, R. Tao, and Q. Du, "Graph information aggregation cross-domain few-shot learning for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jun. 30, 2022, doi: 10.1109/TNNLS.2022.3185795.

[62] Y. Zhang, M. Zhang, W. Li, S. Wang, and R. Tao, "Language-aware domain generalization network for cross-scene hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5501312.

[63] Y. Xu, B. Du, and L. Zhang, "Robust self-ensembling network for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Aug. 19, 2022, doi: 10.1109/TNNLS.2022.3198142.

[64] S. Hao, W. Wang, and M. Salzmann, "Geometry-aware deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2448–2460, Mar. 2021.

[65] Y. Zhang, W. Li, M. Zhang, Y. Qu, R. Tao, and H. Qi, "Topological structure and semantic information transfer network for cross-scene hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 6, pp. 2817–2830, Jun. 2023.

[66] Y. Zhang, W. Li, W. Sun, R. Tao, and Q. Du, "Single-source domain expansion network for cross-scene hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 32, pp. 1498–1512, 2023.

[67] R. Thoreau, V. Achard, L. Risser, B. Berthelot, and X. Briottet, "Active learning for hyperspectral image classification: A comparative review," *IEEE Geosci. Remote Sens. Mag. Replaces Newsletter*, vol. 10, no. 3, pp. 256–278, Sep. 2022.

[68] F. Shi, Z. Wang, M. Hu, and G. Zhai, "Active learning plus deep learning can establish cost-effective and robust model for multichannel image: A case on hyperspectral image classification," *Sensors*, vol. 20, no. 17, p. 4975, 2020.

[69] M. Xu, Q. Zhao, and S. Jia, "Multiview spatial–spectral active learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5512415.

[70] P. Liu, H. Zhang, and K. B. Eom, "Active deep learning for classification of hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 10, no. 2, pp. 712–724, Feb. 2017.

[71] J. M. Haut, M. E. Paoletti, J. Plaza, J. Li, and A. Plaza, "Active learning with convolutional neural networks for hyperspectral image classification using a new Bayesian approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 11, pp. 6440–6461, Nov. 2018.

[72] U. Patel, H. Dave, and V. Patel, "Hyperspectral image classification using uncertainty and diversity based active learning," *Scalable Computing, Pract. Exper.*, vol. 22, no. 3, pp. 283–293, Nov. 2021.

[73] Q. Wang, M. Chen, J. Zhang, S. Kang, and Y. Wang, "Improved active deep learning for semi-supervised classification of hyperspectral image," *Remote Sens.*, vol. 14, no. 1, p. 171, Dec. 2021.

[74] G. Wang and P. Ren, "Hyperspectral image classification with feature-oriented adversarial active learning," *Remote Sens.*, vol. 12, no. 23, p. 3879, Nov. 2020.

[75] D. Yoo and I. S. Kweon, "Learning loss for active learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 93–102.

[76] Z. Lei, Y. Zeng, P. Liu, and X. Su, "Active deep learning for hyperspectral image classification with uncertainty learning," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, 2022, Art. no. 5502405.

[77] B. Liu et al., "Active deep densely connected convolutional network for hyperspectral image classification," *Int. J. Remote Sens.*, vol. 42, no. 15, pp. 5915–5934, Aug. 2021.

[78] G. Hinton, "How to represent part-whole hierarchies in a neural network," 2021, *arXiv:2102.12627*.

[79] A. Kirsch, J. van Amersfoort, and Y. Gal, "BatchBALD: Efficient and diverse batch acquisition for deep Bayesian active learning," 2019, *arXiv:1906.08158*.

[80] O. Sener and S. Savarese, "Active learning for convolutional neural networks: A core-set approach," 2017, *arXiv:1708.00489v4*.

[81] S. Sinha, S. Ebrahimi, and T. Darrell, "Variational adversarial active learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 5971–5980.

[82] K. Konyushkova, R. Sznitman, and P. Fua, "Learning active learning from data," 2017, *arXiv:1703.03365*.

[83] B. Demir, C. Persello, and L. Bruzzone, "Batch-mode active-learning methods for the interactive classification of remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 3, pp. 1014–1031, Mar. 2011.



Heng Wang received the B.S. degree in computer and communication engineering from the Zhengzhou University of Light Industry, Zhengzhou, China, in 2015, and the M.S. degree in information and communication engineering from Harbin Engineering University, Harbin, China, in 2019, where he is currently pursuing the Ph.D. degree with the College of Information and Communication Engineering.

His research interests include computer vision, deep learning, remote sensing, and deep generative models.



Liguo Wang (Member, IEEE) received the M.S. degree in natural science and the Ph.D. degree in engineering from the Harbin Institute of Technology (HIT), Harbin, China, in 2002 and 2006, respectively.

From 2006 to 2008, he held a post-doctoral research position at the College of Information and Communications Engineering, Harbin Engineering University, Harbin, where he is currently a Professor. He has published four books, 35 patents, and more than 230 papers in journals and conference proceedings. His research interests are remote sensing, image processing, and machine learning.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 05:19:55 UTC from IEEE Xplore. Restrictions apply.